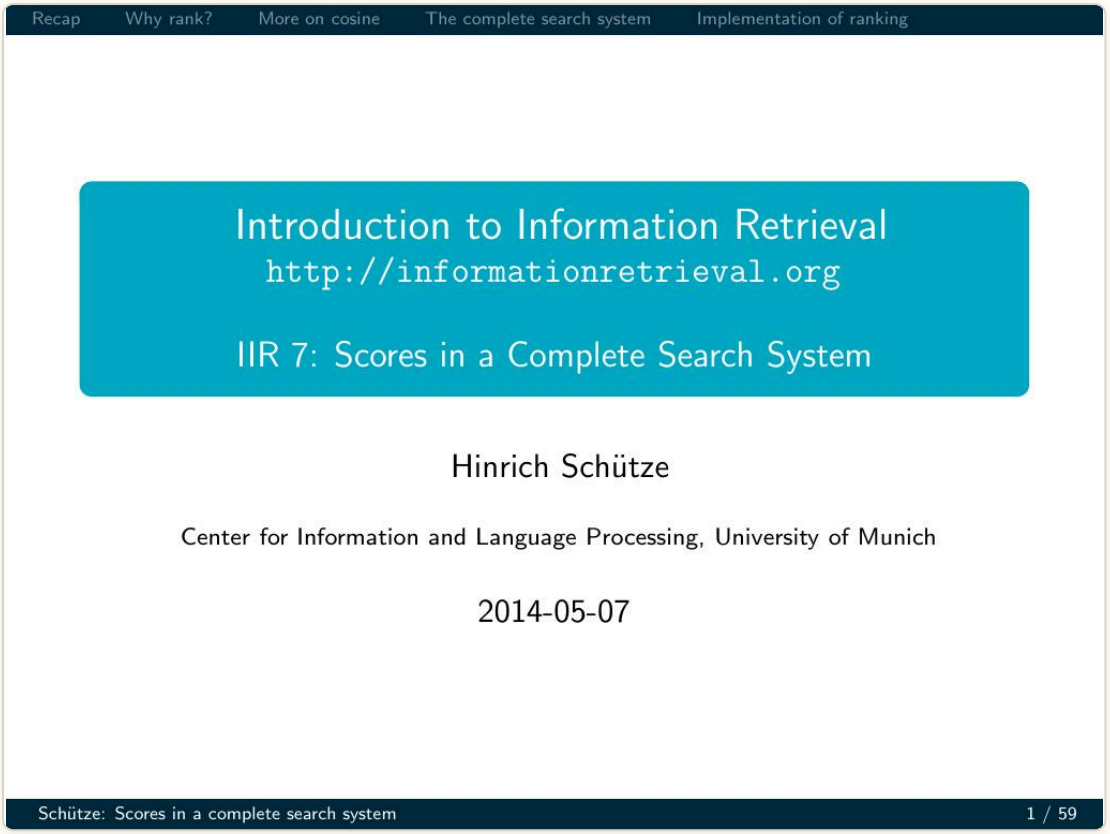
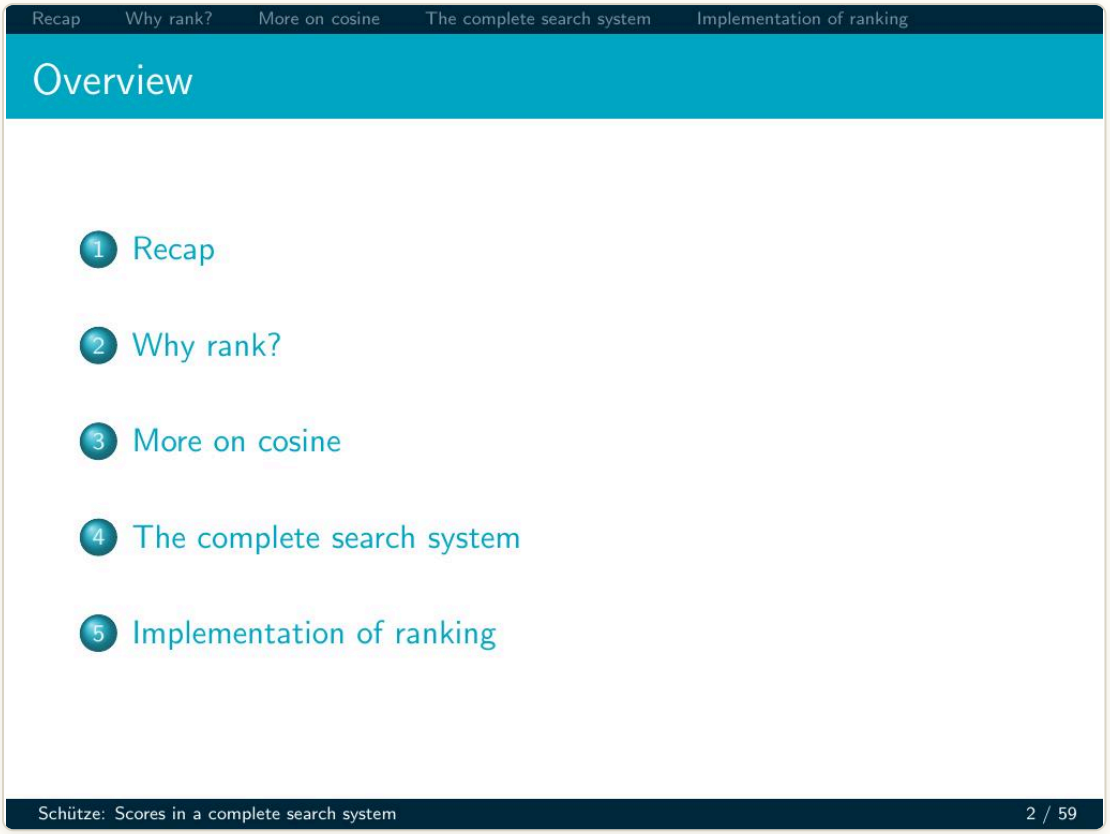


정보검색 입문 — IIR 7: 완전한 검색 시스템에서의 점수

- Hinrich Schütze, 뮌헨대 (2014-05-07)

✍️ 메모

💡 이해 7강 핵심: 순위가 왜 중요한가(구글 사용자 연구) → 피벗 정규화 → 계층 색인·완전한 검색 시스템 → top-k (최소 힙, 조기 종료, 누산기)

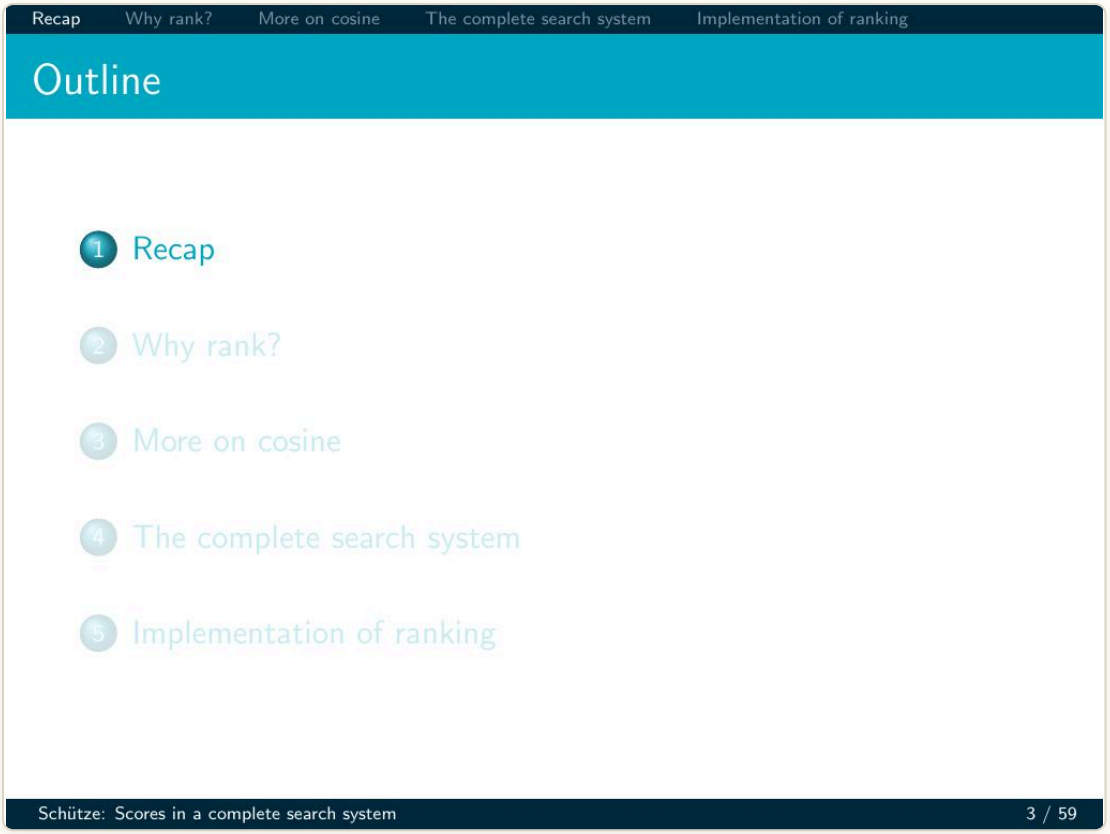


개요

- ① 복습 ② 왜 순위인가? ③ 코사인 더 보기 ④ 완전한 검색 시스템 ⑤ 순위 구현

✍ 메모

— 넘어가도 되는 쪽

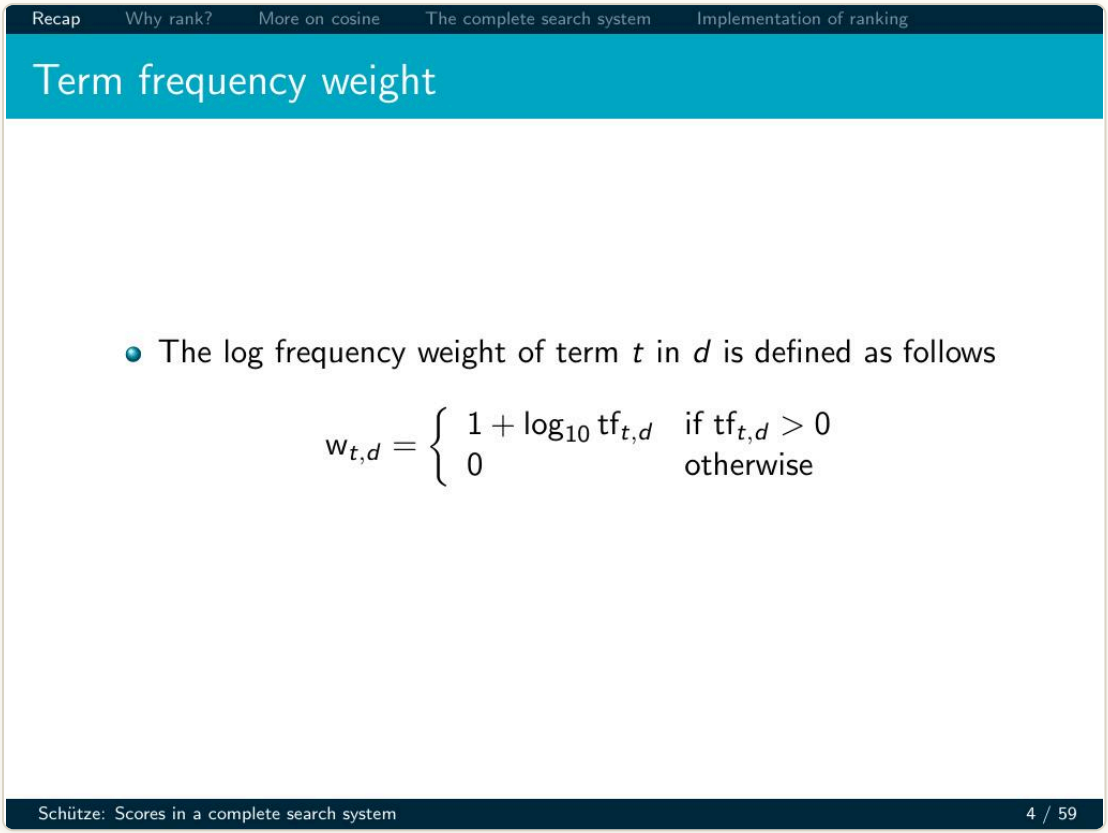


목차

- 복습 → 왜 순위가 중요한가 → 코사인 더 보기 → 완전한 검색 시스템 → 순위 구현
- 지금: 복습

✍ 메모

— 넘어가도 되는 쪽

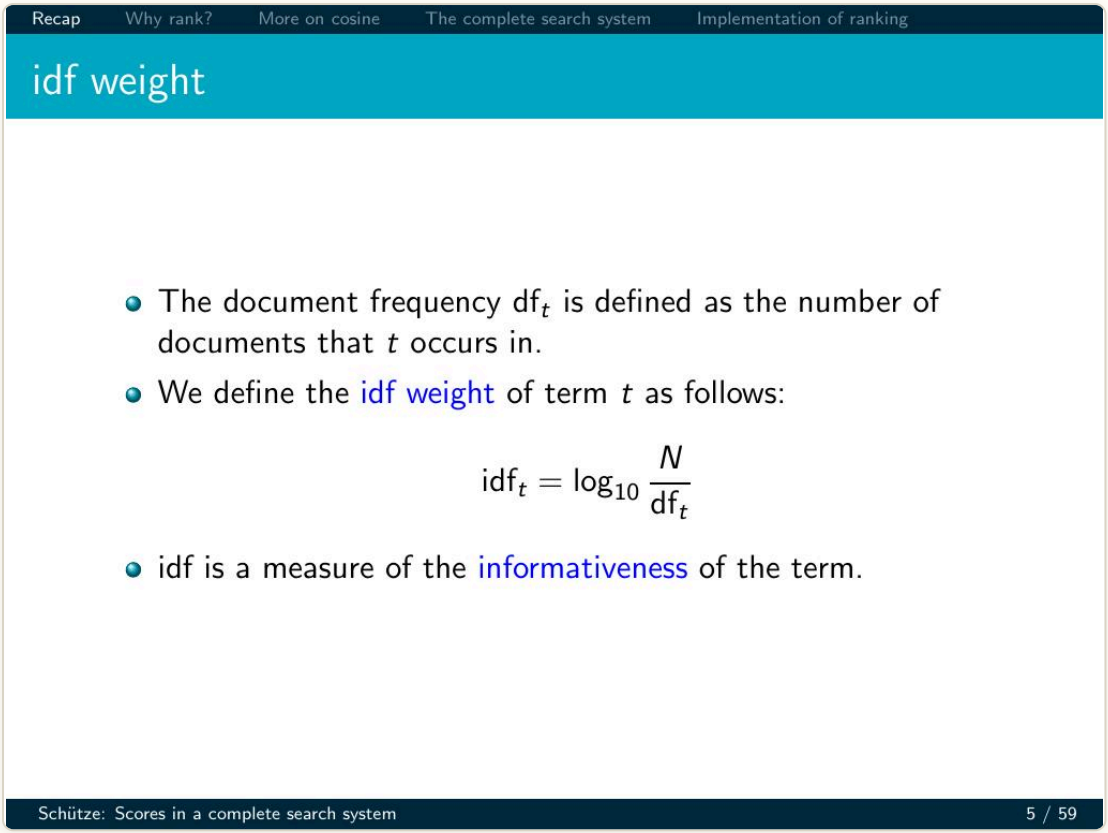


복습: 단어 빈도 가중치

- w(t,d) = 1 + $\log_{10}$ tf(t,d) (tf > 0), 아니면 0

✍ 메모

— 넘어가도 되는 쪽

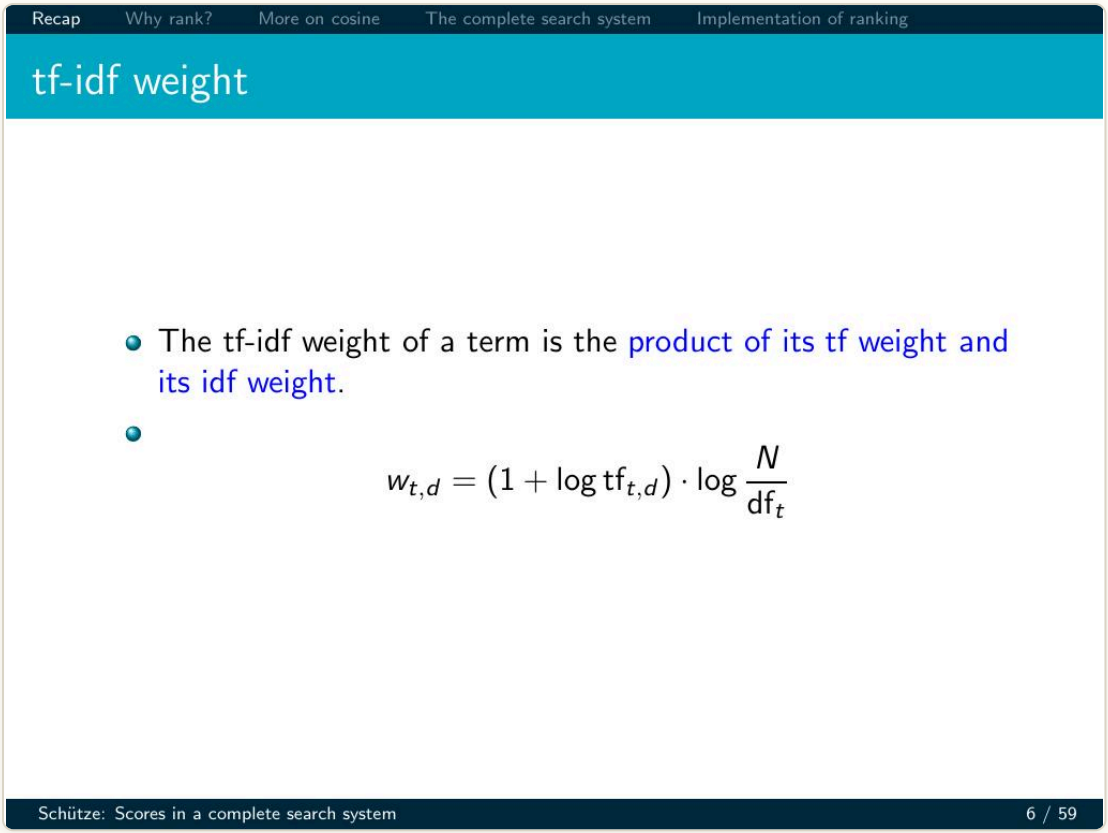


복습: idf 가중치

- df(t) = 단어 t가 나오는 문서 수
- idf(t) = $\log_{10}$ (N / df(t)) — 단어 정보량의 척도

✍ 메모

— 넘어가도 되는 쪽



복습: tf-idf 가중치

- w(t,d) = (1 + log tf(t,d)) · log (N / df(t))

✍ 메모

— 넘어가도 되는 쪽

Cosine similarity between query and document

$$\cos(\vec{q},\vec{d}) = \text{SIM}(\vec{q},\vec{d}) = \frac{\vec{q}}{|\vec{q}|}\cdot\frac{\vec{d}}{|\vec{d}|} = \sum_{i=1}^{|V|}\frac{q_i}{\sqrt{\sum_{i=1}^{|V|}q_i^2}}\cdot\frac{d_i}{\sqrt{\sum_{i=1}^{|V|}d_i^2}}$$

- q_i is the tf-idf weight of term i in the query.
- d_i is the tf-idf weight of term i in the document.
- $|\vec{q}|$ and $|\vec{d}|$ are the lengths of $\vec{q}$ and $\vec{d}$.
- $\vec{q}/|\vec{q}|$ and $\vec{d}/|\vec{d}|$ are length-1 vectors (= normalized).

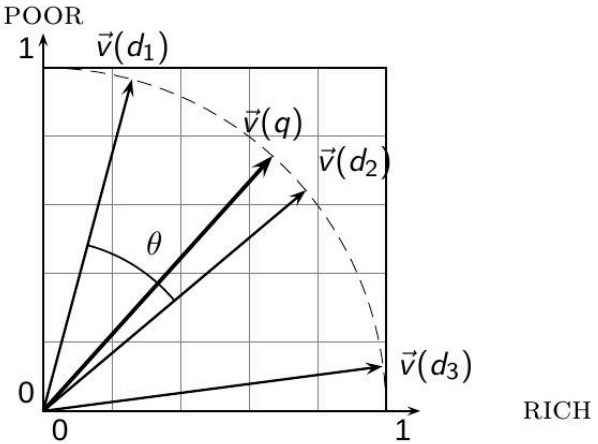
복습: 질의와 문서의 코사인 유사도

- cos(q, d) = Σ $q_i d_i$ / (|q| |d|)
- q_i, d_i = 질의·문서에서 단어 i의 tf-idf 가중치
- q/|q|와 d/|d|는 길이 1인 벡터 (= 정규화)

✍ 메모

— 넘어가도 되는 쪽

Cosine similarity illustrated



복습: 코사인 유사도 (그림)

- RICH-POOR 평면, v(q)와 v(d_2) 사이 각도 θ가 작다

✍ 메모

— 넘어가도 되는 쪽

tf-idf example: lnc.ltn

Query: "best car insurance". Document: "car insurance auto insurance".

word	query					document				product
	tf-raw	tf-wght	df	idf	tf-idf weight	tf-raw	tf-wght	tf-wght	n'lized	
auto	0	0	5000	2.3	0	1	1	1	0.52	0
best	1	1	50000	1.3	1.3	0	0	0	0	0
car	1	1	10000	2.0	2.0	1	1	1	0.52	1.04
insurance	1	1	1000	3.0	3.0	2	1.3	1.3	0.68	2.04

Key to columns: tf-raw: raw (unweighted) term frequency, tf-wght: logarithmically weighted term frequency, df: document frequency, idf: inverse document frequency, weight: the final weight of the term in the query or document, n'lized: document weights after cosine normalization, product: the product of final query weight and final document weight

$\sqrt{1^2 + 0^2 + 1^2 + 1.3^2} \approx 1.92$
$1/1.92 \approx 0.52$
$1.3/1.92 \approx 0.68$

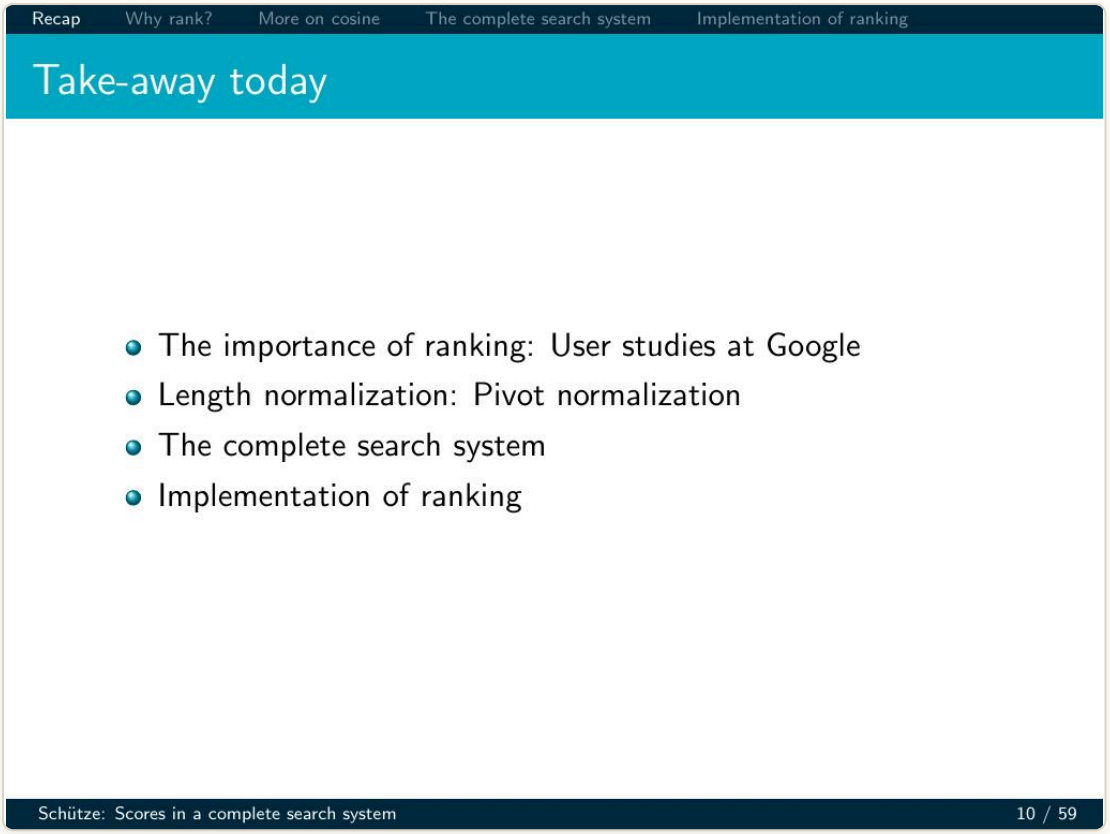
Final similarity score between query and document: $\sum_i w_{qi} \cdot w_{di} = 0 + 0 + 1.04 + 2.04 = 3.08$

복습: tf-idf 예 lnc.ltn

- 질의 "best car insurance", 문서 "car insurance auto insurance"
- 문서 길이 √(1² + 0² + 1² + 1.3²) ≈ 1.92 → 정규화 0.52, 0, 0.52, 0.68
- 최종 점수 = 0 + 0 + 1.04 + 2.04 = **3.08**

✍ 메모

— 넘어가도 되는 쪽

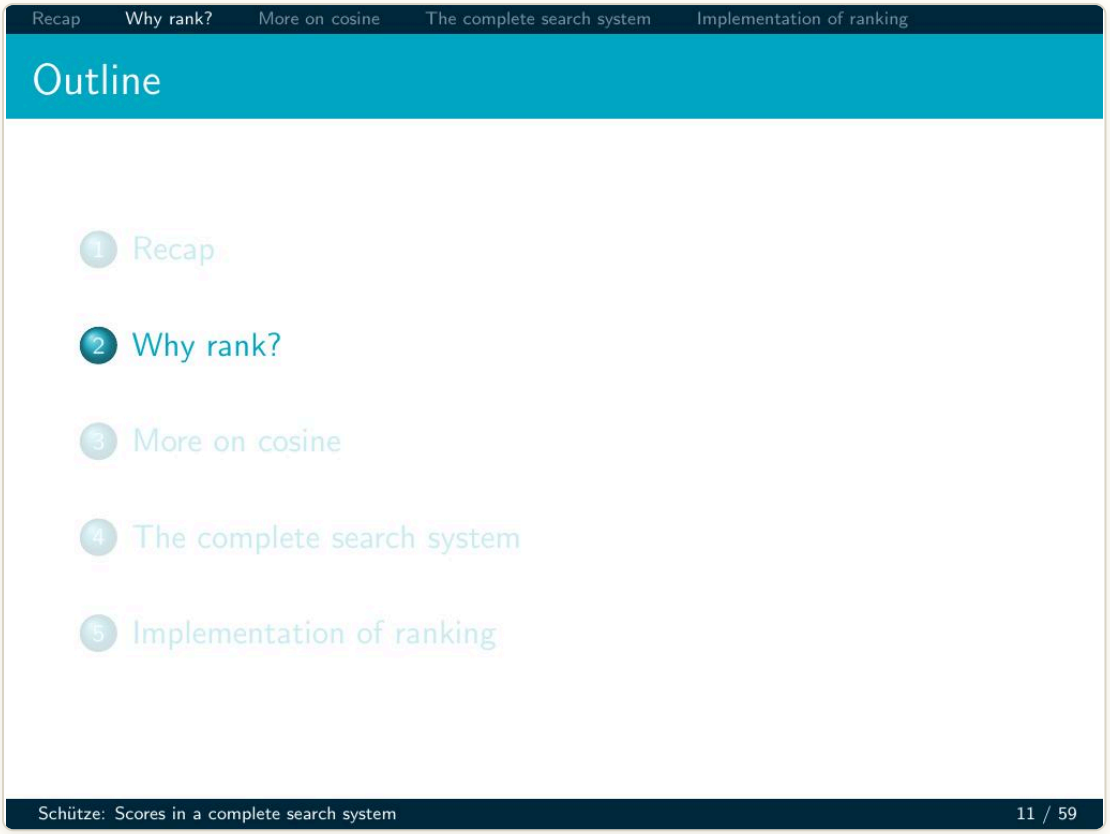


오늘 가져갈 것

- **순위의 중요성**: 구글의 사용자 연구
- **길이 정규화**: 피벗 정규화
- **완전한 검색 시스템**
- **순위 구현**

✍️ 메모

— 넘어가도 되는 쪽

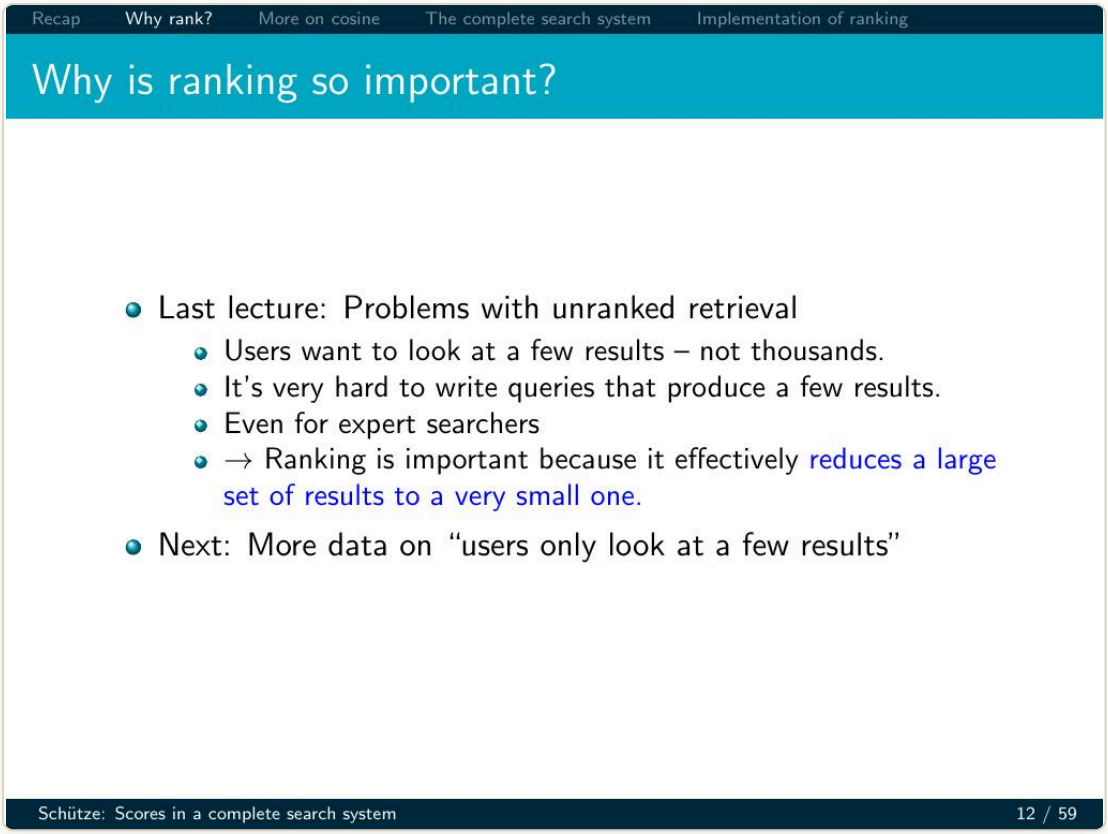


목차

- 복습 → 왜 순위가 중요한가 → 코사인 더 보기 → 완전한 검색 시스템 → 순위 구현
- 지금: **왜 순위인가?**

✍️ 메모

— 넘어가도 되는 쪽

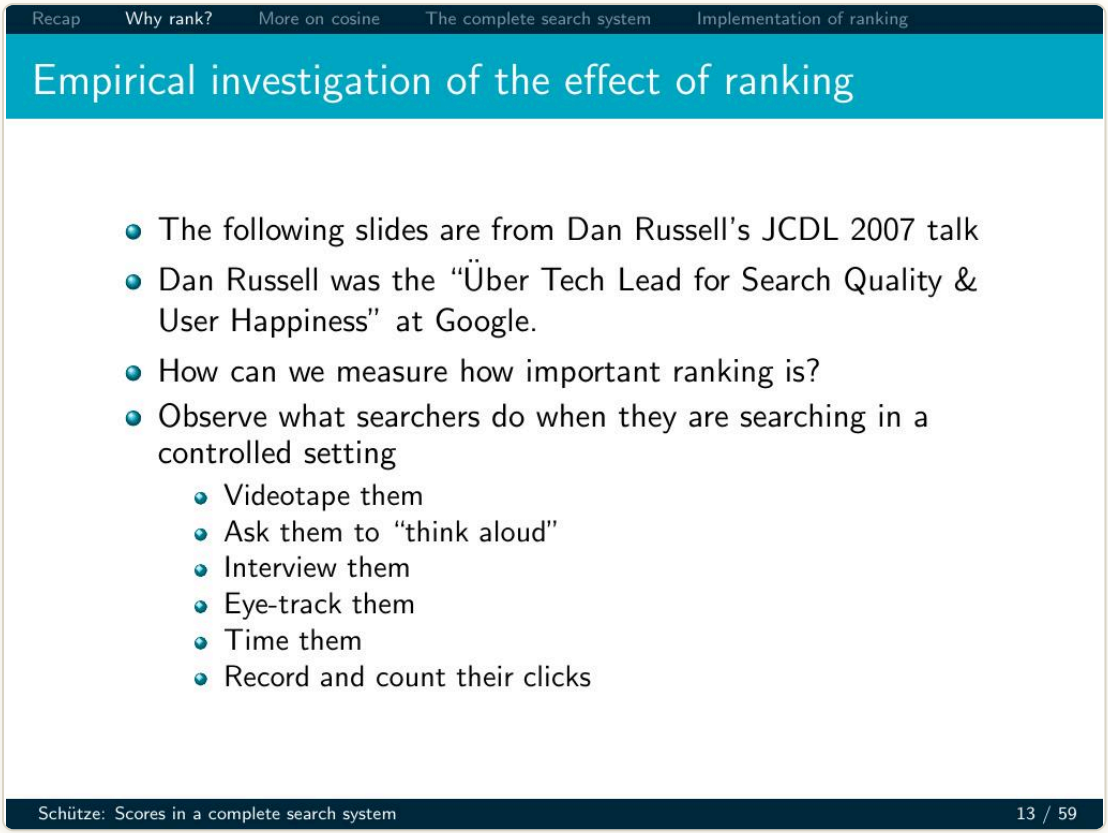


순위는 왜 그렇게 중요한가?

- 지난 강의: 순위 없는 검색의 문제
  - 사용자는 결과를 **몇 개만** 보고 싶다 — 수천 개가 아니라
  - 결과가 몇 개만 나오는 질의를 쓰기는 **매우 어렵다**
    - 전문 검색자에게도
- → 순위는 큰 결과 집합을 **아주 작게 줄여 주기** 때문에 중요하다
- 다음: '사용자는 결과를 몇 개만 본다'에 대한 데이터 더 보기

✍ 메모

— 넘어가도 되는 쪽

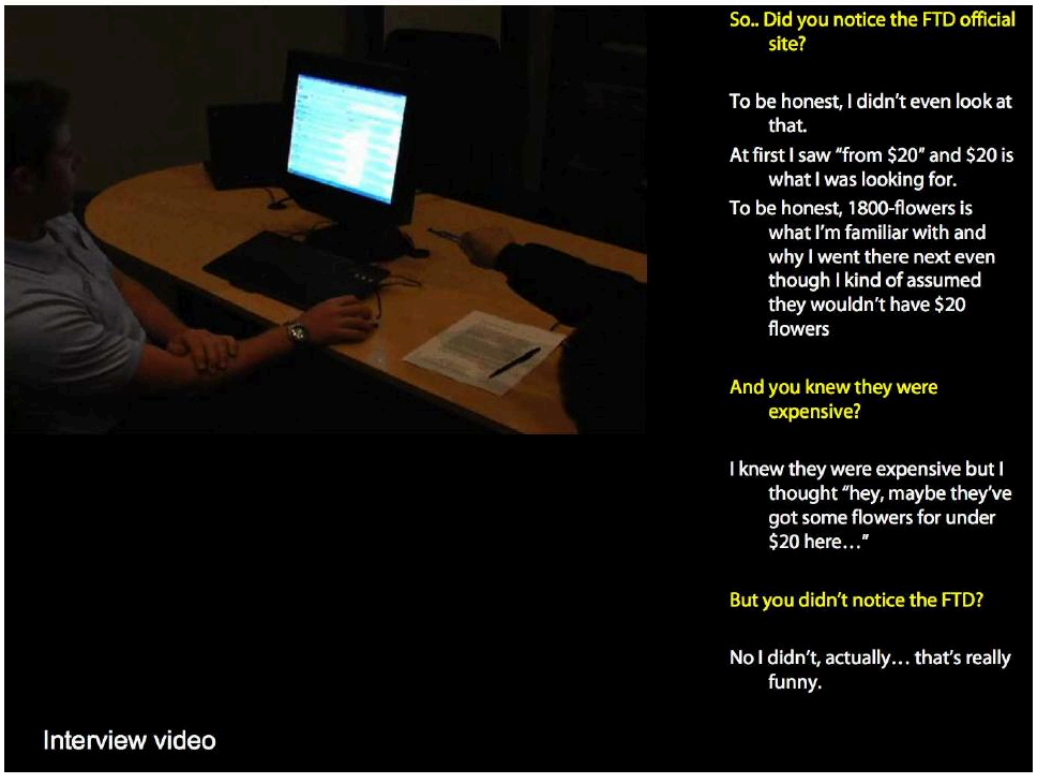


순위 효과에 대한 경험적 조사

- 다음 쪽들은 Dan Russell의 JCDL 2007 발표에서
- Dan Russell은 구글의 '검색 품질·사용자 만족 총괄 기술 리더'
- 순위가 얼마나 중요한지 어떻게 잴까?
  - 통제된 환경에서 검색하는 사람을 **관찰**
  - 비디오 녹화 / 소리 내어 생각하기(think aloud) / 인터뷰
  - **시선 추적** / 시간 재기 / **클릭** 기록·집계

✍️ 메모

— 넘어가도 되는 쪽

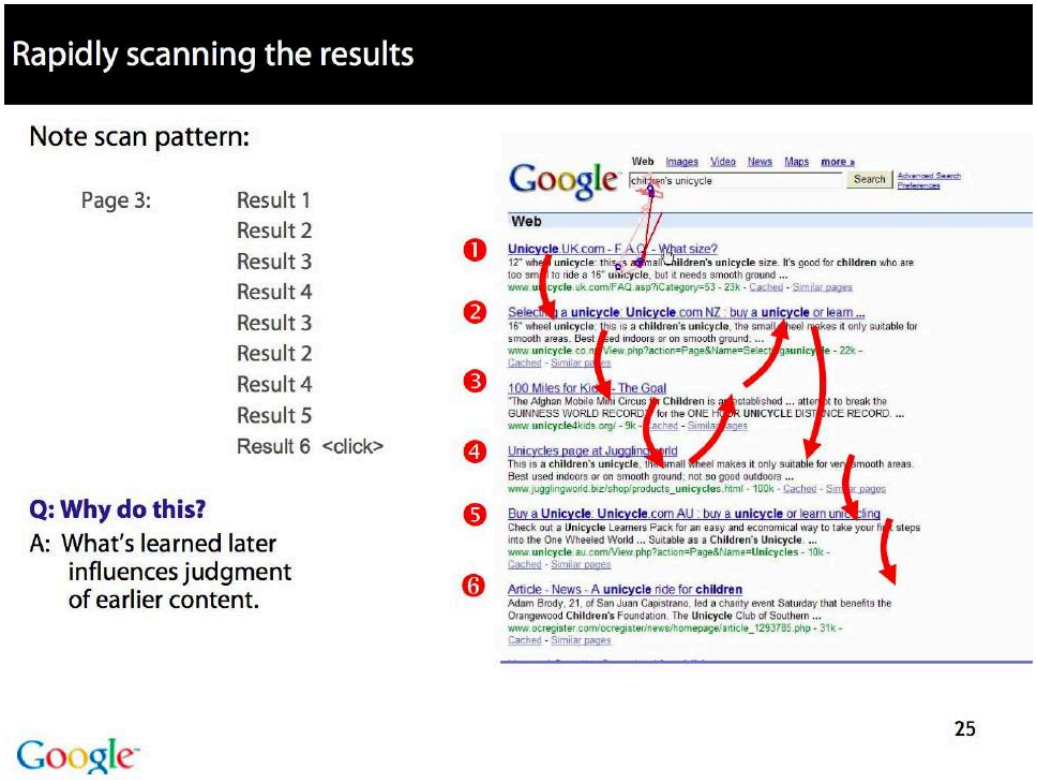


구글 사용자 연구: 인터뷰 영상

- 진행자: FTD 공식 사이트는 보셨어요? — 참가자: 솔직히 안 봤어요. '$20부터'가 먼저 보였고 $20짜리를 찾고 있었거든요 …
- → 사용자는 결과를 꼼꼼히 다 읽지 않고 **눈에 띄는 것**만 본다

✍ 메모

— 넘어가도 되는 쪽

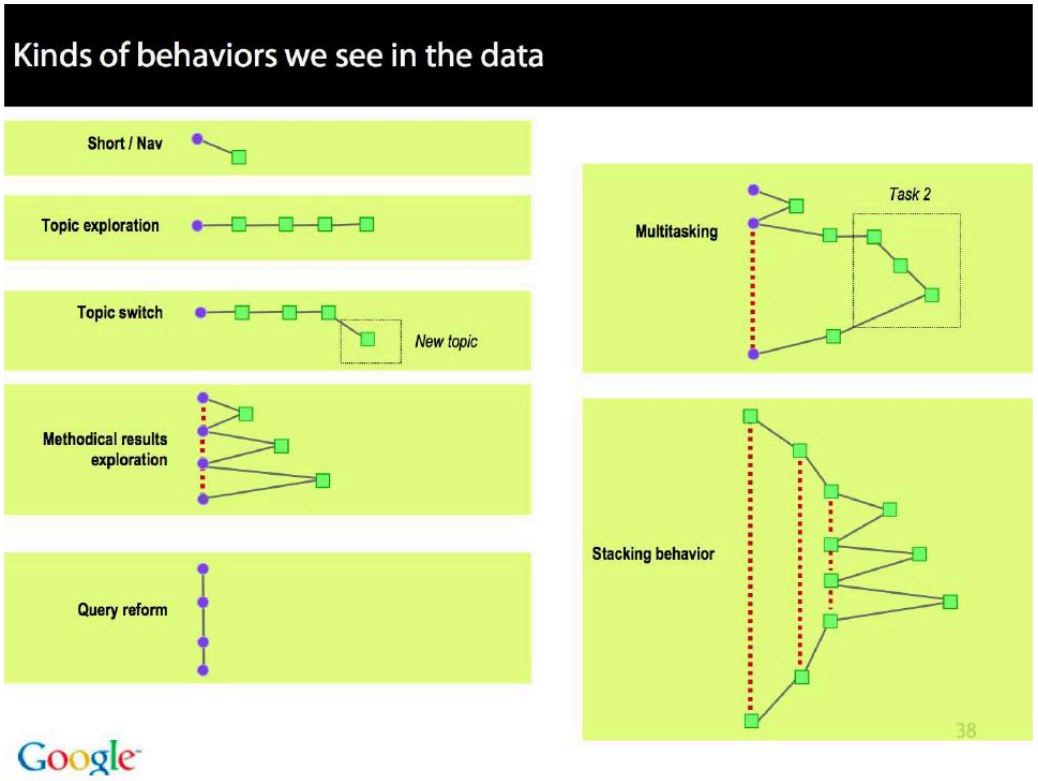


구글 사용자 연구: 결과를 빠르게 훑어보기

- 3페이지 훑는 순서: 결과 1 → 2 → 3 → 4 → 3 → 2 → 4 → 5 → 6 <클릭>
- 왜 이렇게 할까? — 나중에 본 내용이 앞의 내용에 대한 판단에 영향을 주기 때문

✍ 메모

— 넘어가도 되는 쪽

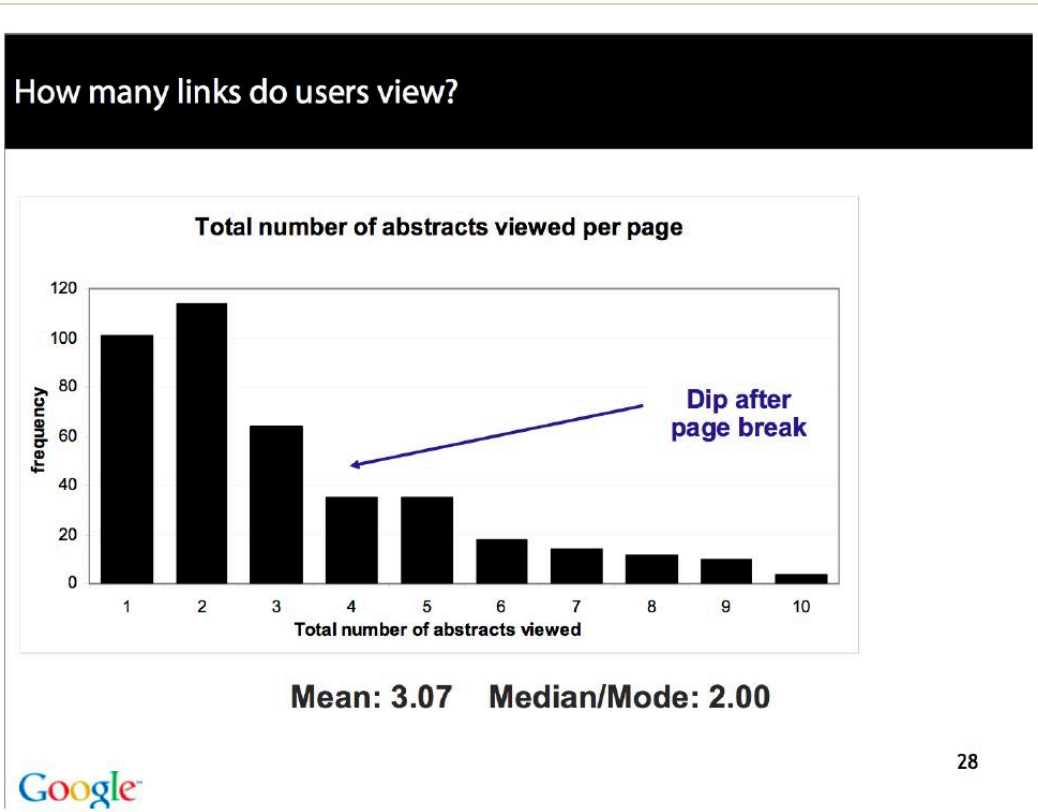


구글 사용자 연구: 데이터에서 보이는 행동 유형

- 짧은 탐색(Short/Nav), 주제 탐색, 주제 전환, 체계적 결과 탐색, 질의 수정, 멀티태스킹, 쌓기(stacking) 행동

✍ 메모

— 넘어가도 되는 쪽

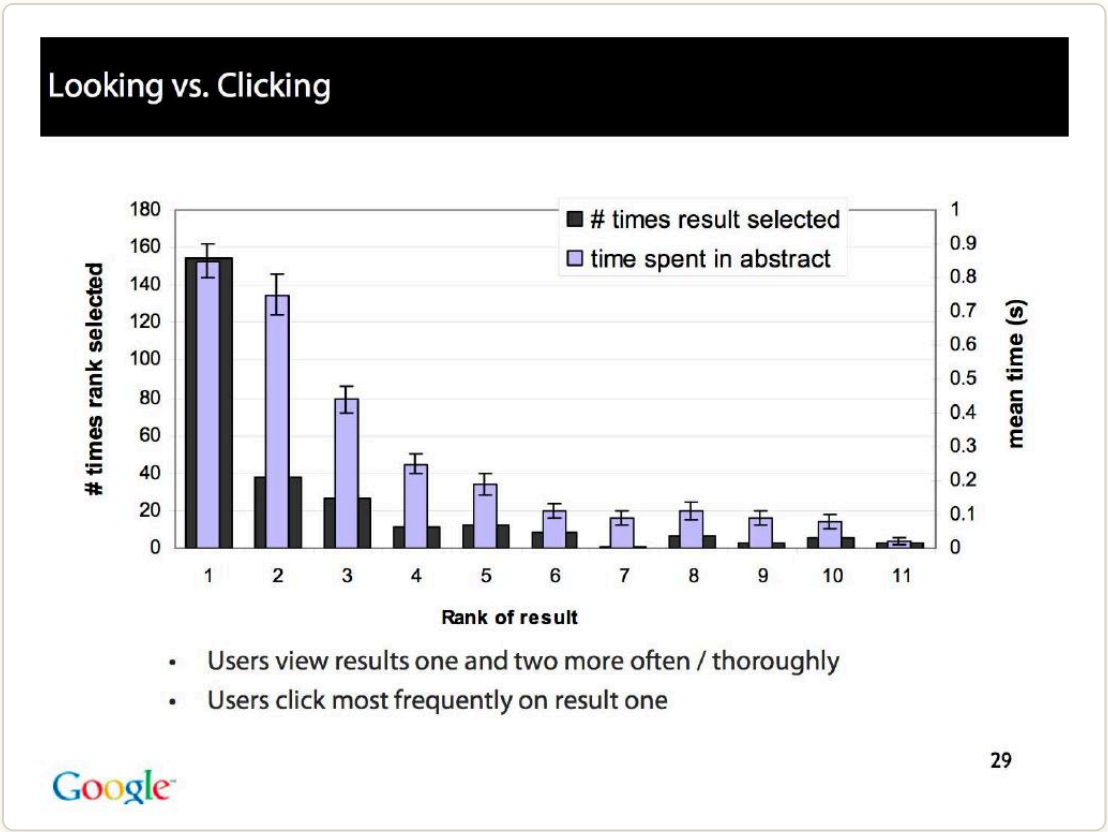


구글 사용자 연구: 사용자는 링크를 몇 개나 보나?

- 페이지당 본 요약(abstract) 수의 분포: 1~2개가 가장 많고, 4개째부터 뚝 떨어짐 (**페이지 구분선 뒤에 감소**)
- 평균 **3.07**, 중앙값·최빈값 **2.00**

✍️ 메모

★ 시험 사용자는 평균 3개 정도만 본다

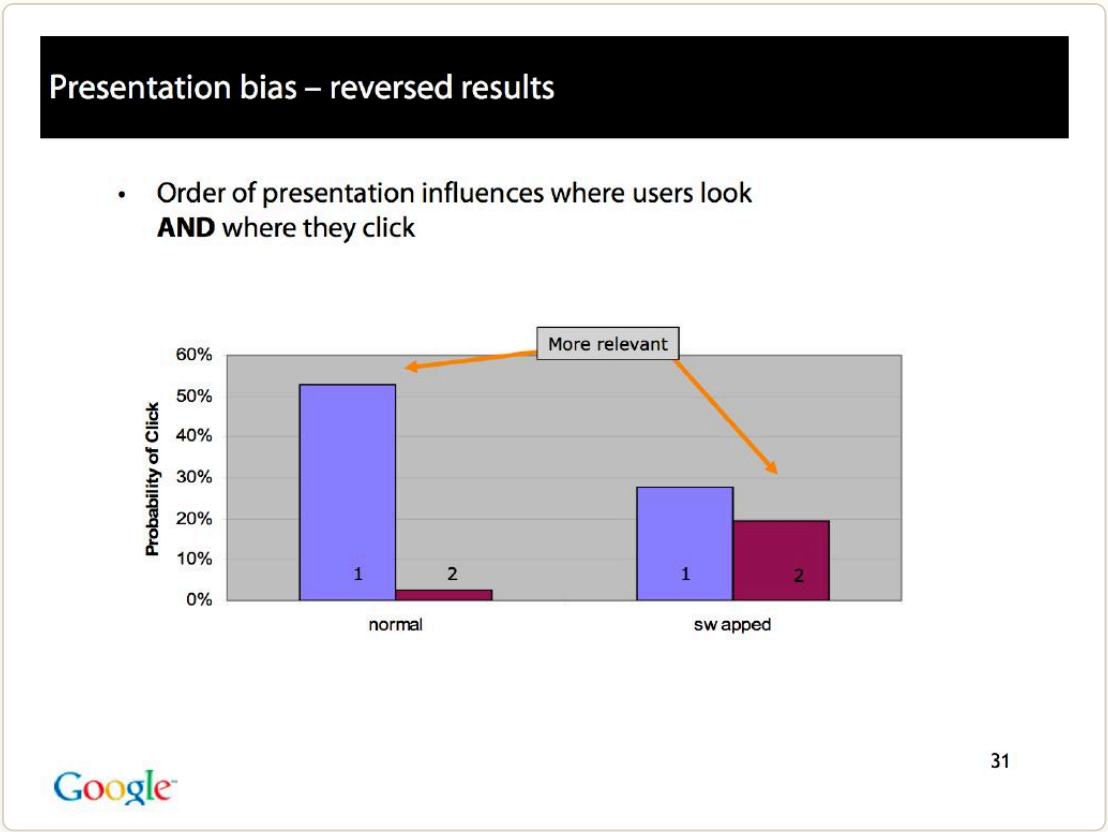


구글 사용자 연구: 보기 vs 클릭

- 순위별 '선택된 횟수'와 '요약을 본 시간'
- 사용자는 결과 1, 2를 더 자주·꼼꼼히 본다
- 사용자는 **결과 1을 가장 자주 클릭**한다

✍ 메모

— 넘어가도 되는 쪽

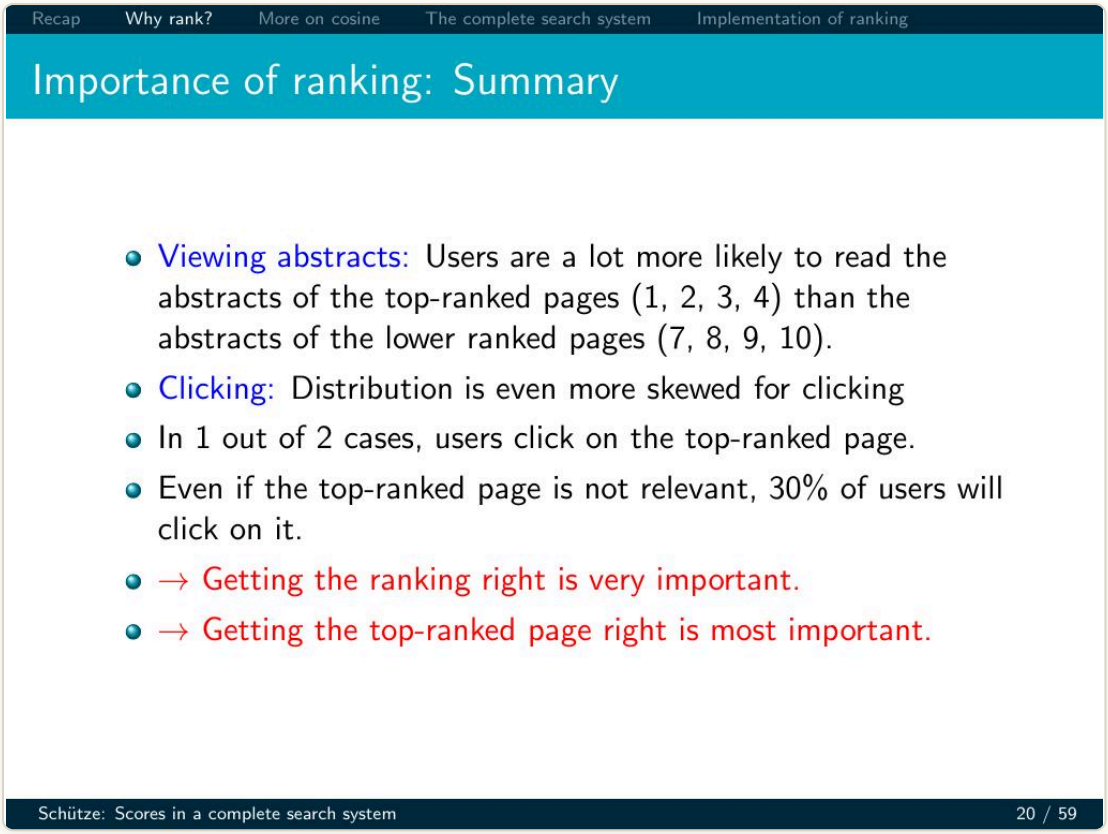


구글 사용자 연구: 제시 편향 — 결과 순서 뒤집기

- 제시 **순서**가 사용자가 **어디를 보고 어디를 클릭할지**에 영향을 준다
- 정상: 1위 클릭 약 50%+, 2위 거의 0 / 1·2위를 **바꿔도**(더 관련 있는 걸 2위로) 1위 클릭이 여전히 많다

✍️ 메모

★ 시험 제시 편향(presentation bias): 순위 자체가 클릭을 끈다

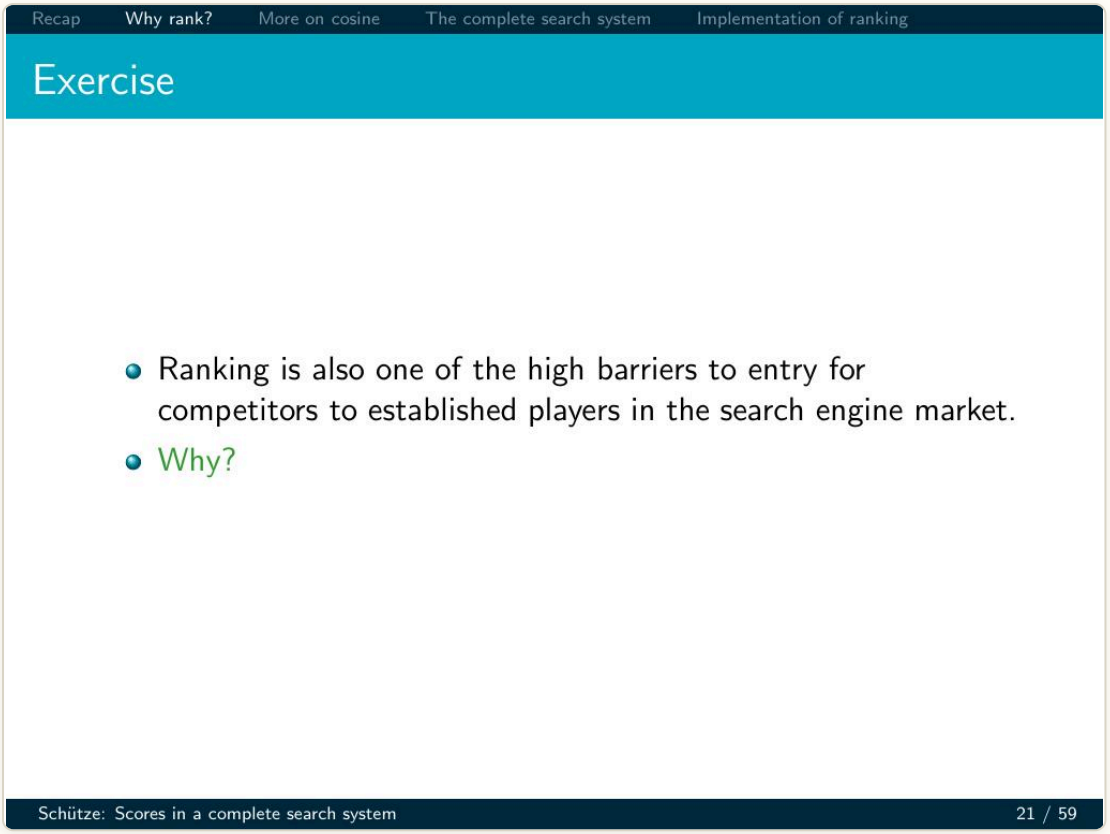


순위의 중요성: 요약

- **요약 보기**: 사용자는 하위(7, 8, 9, 10)보다 상위(1, 2, 3, 4) 페이지의 요약을 훨씬 많이 읽는다
- **클릭**: 분포가 더 치우친다
  - **2번 중 1번**은 1위 페이지를 클릭
  - 1위가 관련 없어도 **30%**는 클릭
- → 순위를 맞게 하는 게 매우 중요
- → **1위를 맞게 하는 게** 가장 중요

✍️ 메모

★ 시험 숫자: 1위 클릭 50%, 관련 없어도 30%

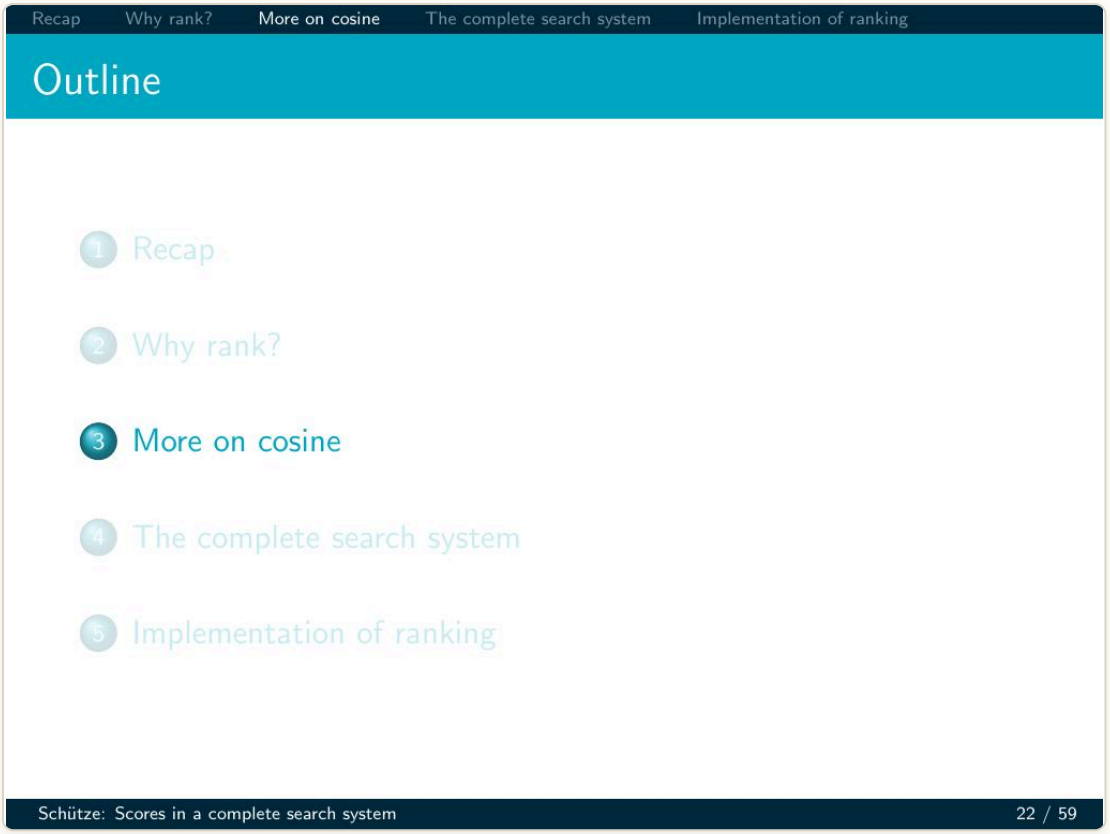


연습

- 순위는 검색 엔진 시장의 기존 강자에 맞서는 경쟁자에게 **높은 진입 장벽** 중 하나이기도 하다. 왜?

✍️ 메모

💡 이해 좋은 순위엔 대량의 클릭·질의 로그와 오랜 튜닝이 필요 — 신규 업체는 그 데이터가 없다



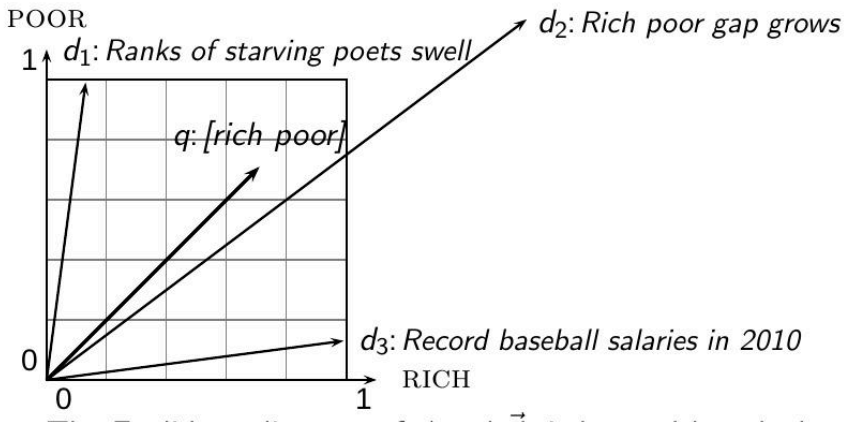
목차

- 복습 → 왜 순위가 중요한가 → 코사인 더 보기 → 완전한 검색 시스템 → 순위 구현
- 지금: **코사인 더 보기**

✍️ 메모

— 넘어가도 되는 쪽

Why distance is a bad idea



The Euclidean distance of $\vec{q}$ and $\vec{d}_2$ is large although the distribution of terms in the query q and the distribution of terms in the document d_2 are very similar.

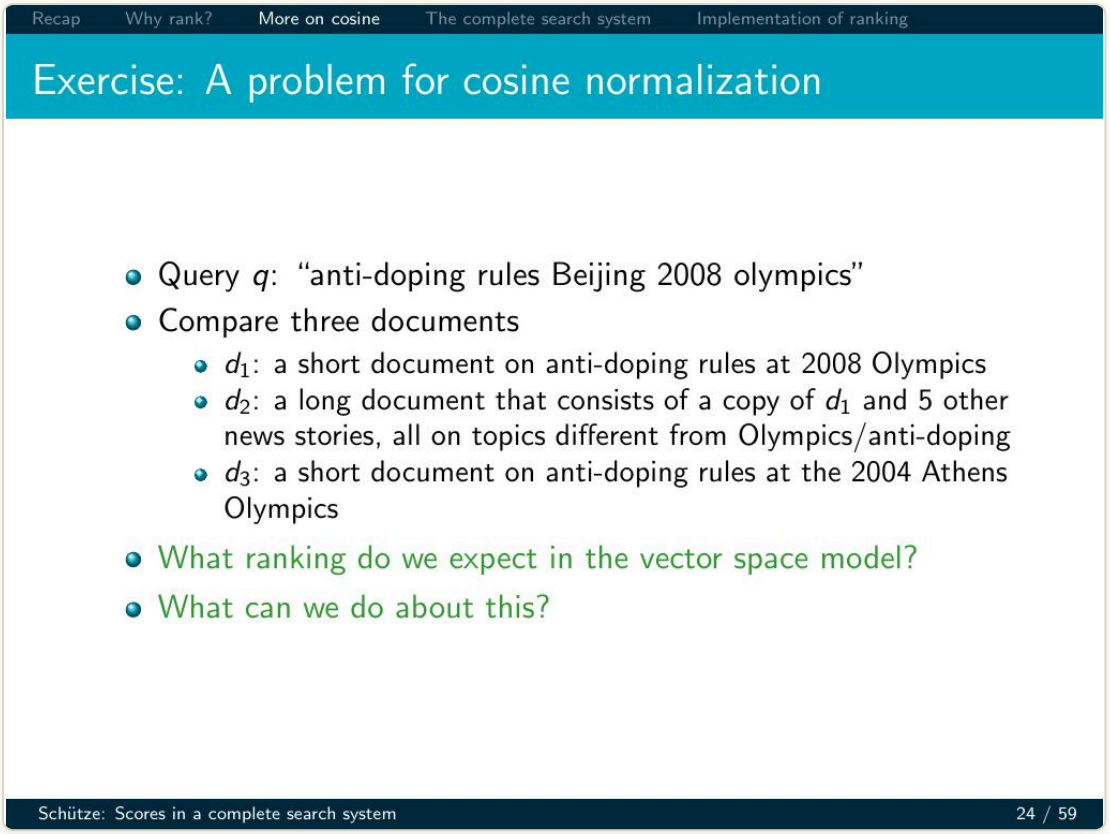
That's why we do length normalization or, equivalently, use cosine to compute query-document matching scores.

거리가 나쁜 이유 (복습)

- q [rich poor]와 d_2 (Rich poor gap grows)의 유클리드 거리는 크지만 단어 분포는 매우 비슷
- 그래서 **길이 정규화**를 하거나, 같은 말로 **코사인**으로 질의-문서 점수를 계산

✍ 메모

— 넘어가도 되는 쪽



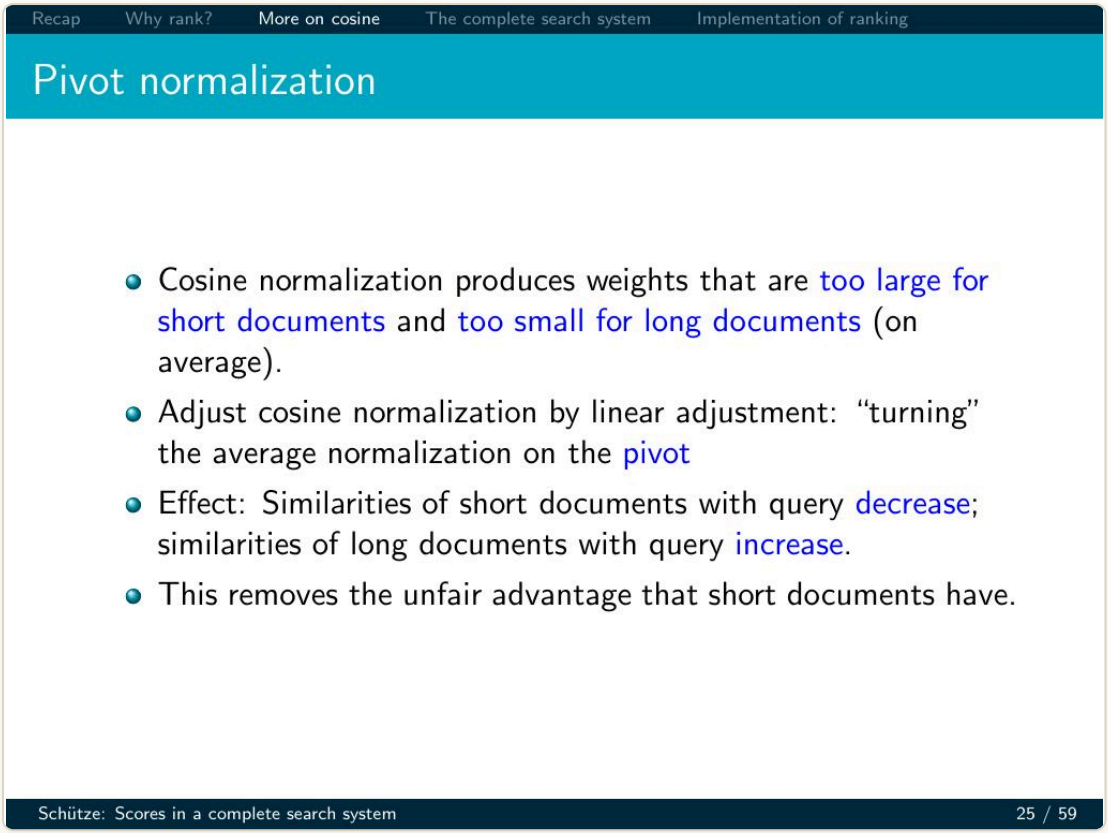
연습: 코사인 정규화의 문제

- 질의 q: "anti-doping rules Beijing 2008 olympics"
- 세 문서 비교
  - d_1: 2008 올림픽 도핑 방지 규정에 관한 **짧은** 문서
  - d_2: d_1 사본 + 올림픽·도핑과 무관한 기사 5개로 된 **긴** 문서
  - d_3: 2004 아테네 올림픽 도핑 방지 규정에 관한 **짧은** 문서
- 벡터 공간 모델에서 어떤 순위를 예상하나?
- 어떻게 하면 좋을까?

✍ 메모

✏ 예·풀이 코사인: $d_1 > d_3 > d_2$ 가능 — d_2는 d_1을 그대로 포함하는데도 길이 때문에 점수가 깎인다

💡 이해 해결: 다음 쪽 피벗 정규화

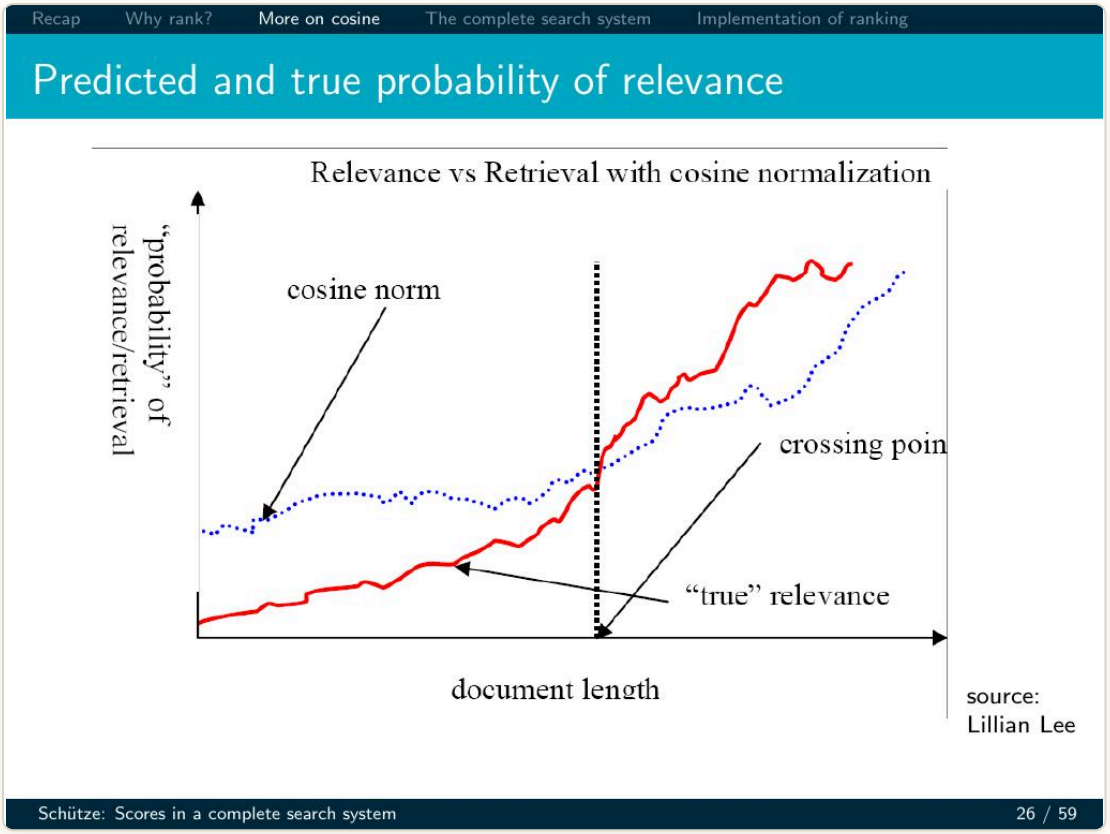


피벗 정규화 (Pivot normalization)

- 코사인 정규화는 (평균적으로) **짧은 문서엔 너무 큰**, **긴 문서엔 너무 작은** 가중치를 만든다
- 코사인 정규화를 **선형 조정**: 평균 정규화 지점(**피벗**)을 중심으로 '돌린다'
- 효과: 짧은 문서와 질의의 유사도는 **감소**, 긴 문서는 **증가**
- 짧은 문서가 가진 **불공정한 이점**을 없앤다

✍ 메모

★ 시험 피벗 정규화 = 짧은 문서 ↓, 긴 문서 ↑

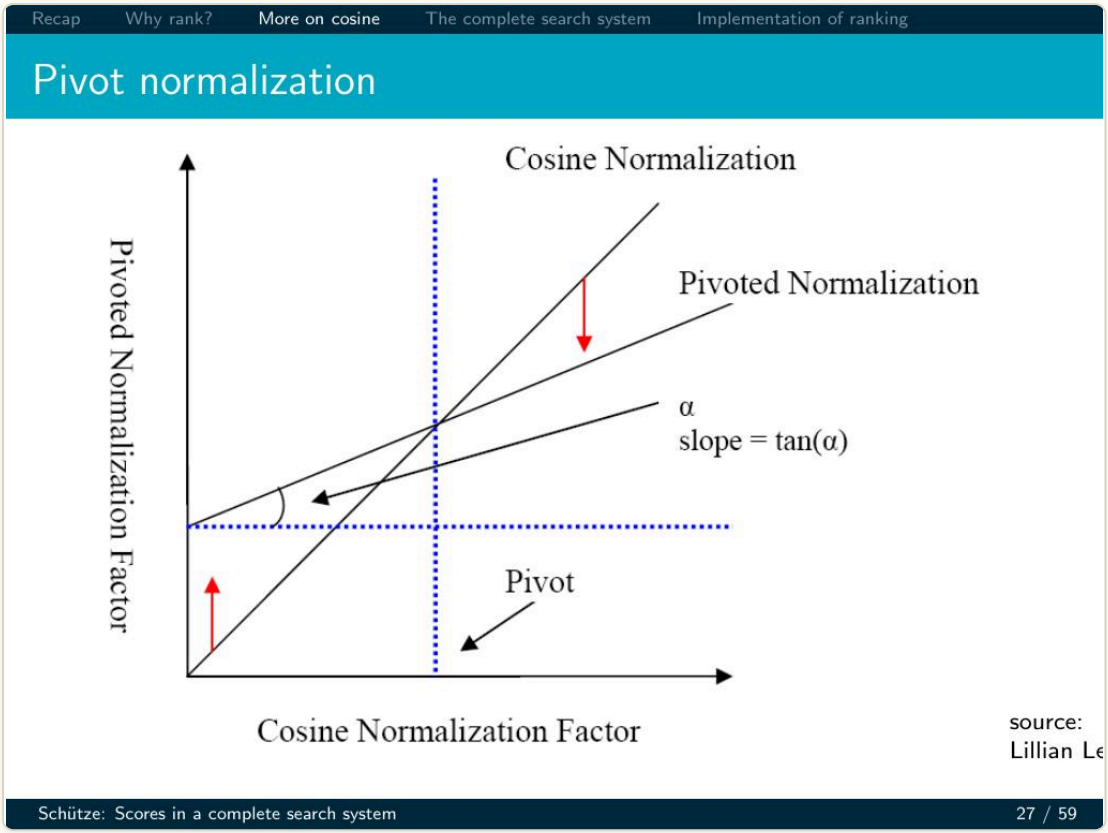


예측된 vs 실제 관련 확률 (그래프)

- x = 문서 길이, y = 관련/검색될 '확률'
- 점선(코사인 정규화로 **검색될** 확률)과 실선('**실제**' 관련 확률)이 **교차점**에서 만난다
- 교차점보다 짧은 문서는 실제보다 많이 검색되고, 긴 문서는 실제보다 적게 검색된다

✍ 메모

— 넘어가도 되는 쪽



피벗 정규화 (그래프)

- x = 코사인 정규화 계수, y = 피벗 정규화 계수
- 피벗 지점에서 두 직선이 교차, 피벗 정규화 직선은 기울기 **tan(α)**로 더 완만
- 피벗보다 왼쪽(짧은 문서): 정규화 계수 ↑ (빨간 화살표 위) → 점수 ↓
- 피벗보다 오른쪽(긴 문서): 정규화 계수 ↓ → 점수 ↑

✍ 메모

— 넘어가도 되는 쪽

Pivoted normalization: Amit Singhal's experiments

Cosine	Pivoted Cosine Normalization				
	Slope				
	0.60	0.65	0.70	**0.75**	0.80
6,526	6,342	6,458	6,574	**6,629**	6,671
0.2840	0.3024	0.3097	0.3144	**0.3171**	0.3162
Improvement	+ 6.5%	+ 9.0%	+10.7%	**+11.7%**	+11.3%

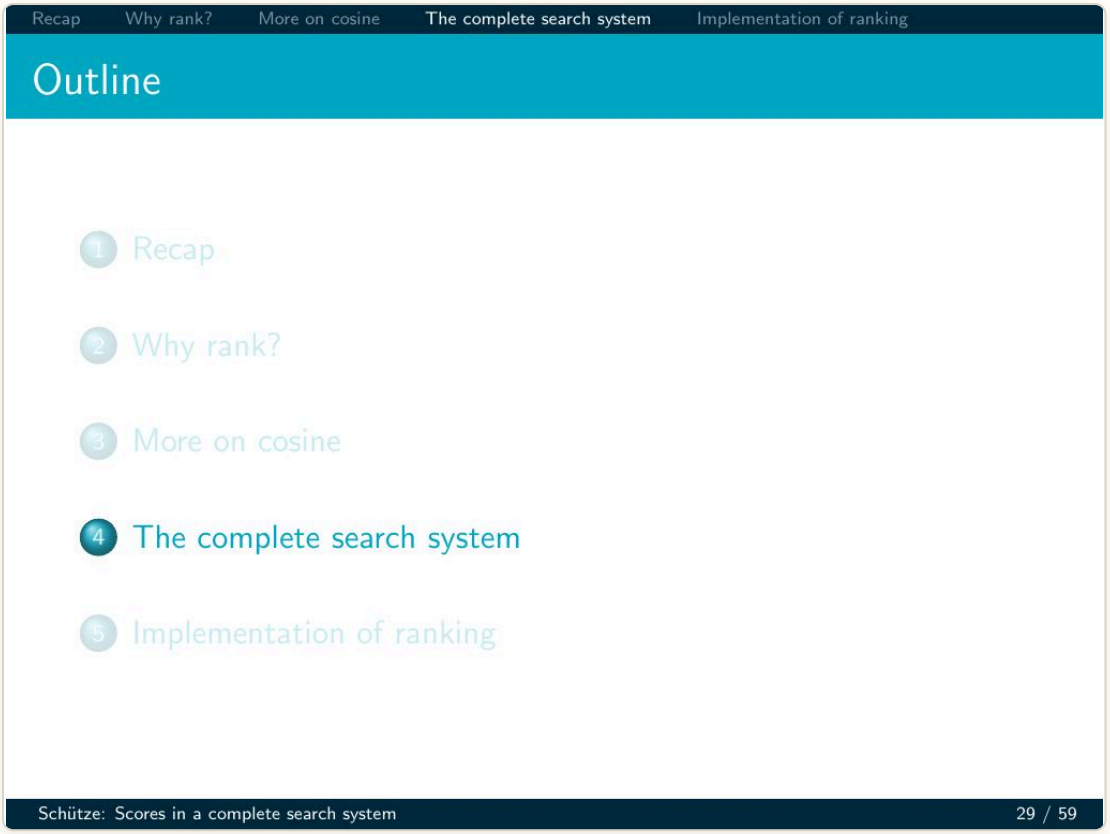
(relevant documents retrieved and (change in) average precision)

피벗 정규화: Amit Singhal의 실험 (표)

- 코사인: 검색된 관련 문서 6,526 / 평균 정밀도 0.2840
- 피벗 코사인 정규화, 기울기별
  - 0.60: 6,342 / 0.3024 (+6.5%)
  - 0.65: 6,458 / 0.3097 (+9.0%)
  - 0.70: 6,574 / 0.3144 (+10.7%)
  - **0.75: 6,629 / 0.3171 (+11.7%)** — 최고
  - 0.80: 6,671 / 0.3162 (+11.3%)

✍ 메모

★ 시험 기울기 0.75에서 평균 정밀도 +11.7%

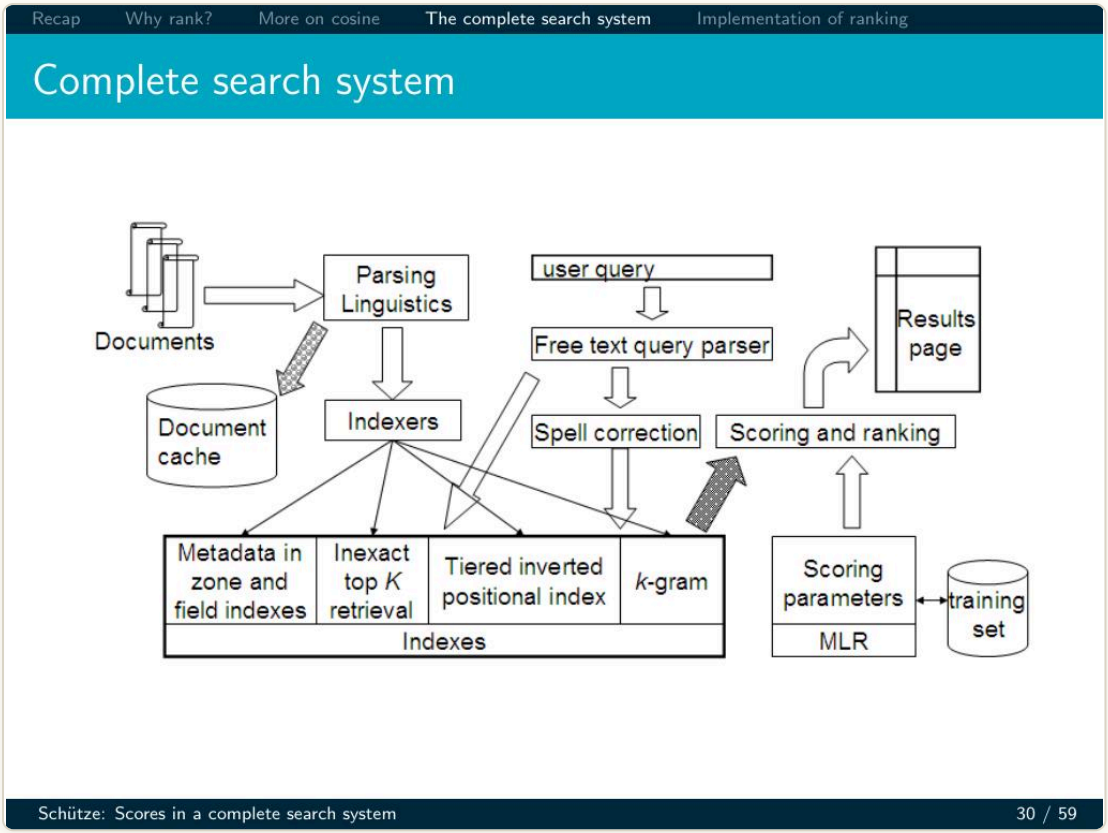


목차

- 복습 → 왜 순위가 중요한가 → 코사인 더 보기 → 완전한 검색 시스템 → 순위 구현
- 지금: **완전한 검색 시스템**

✍️ 메모

— 넘어가도 되는 쪽

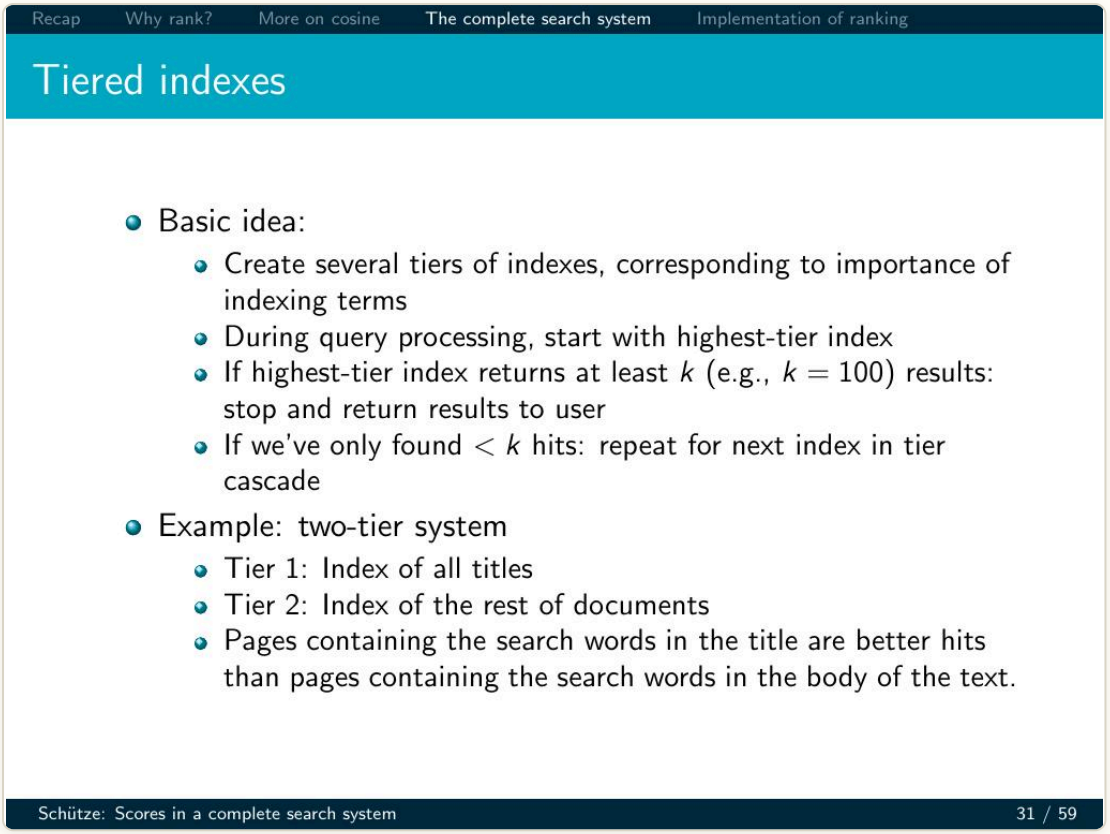


완전한 검색 시스템 (그림)

- 문서 → 파싱·언어처리 → 색인기 → 색인들 (메타데이터·영역/필드 색인, 부정확 top K 검색, 계층 역 위치 색인, k-gram) / 문서 캐시
- 사용자 질의 → 자유 텍스트 질의 파서 → 철자 교정 → 점수화·순위 → 결과 페이지
- 점수 매개변수는 학습 집합으로 **MLR**(기계학습 순위)

✍ 메모

★ 시험 구성 요소 그림 — 빈칸 채우기 대비

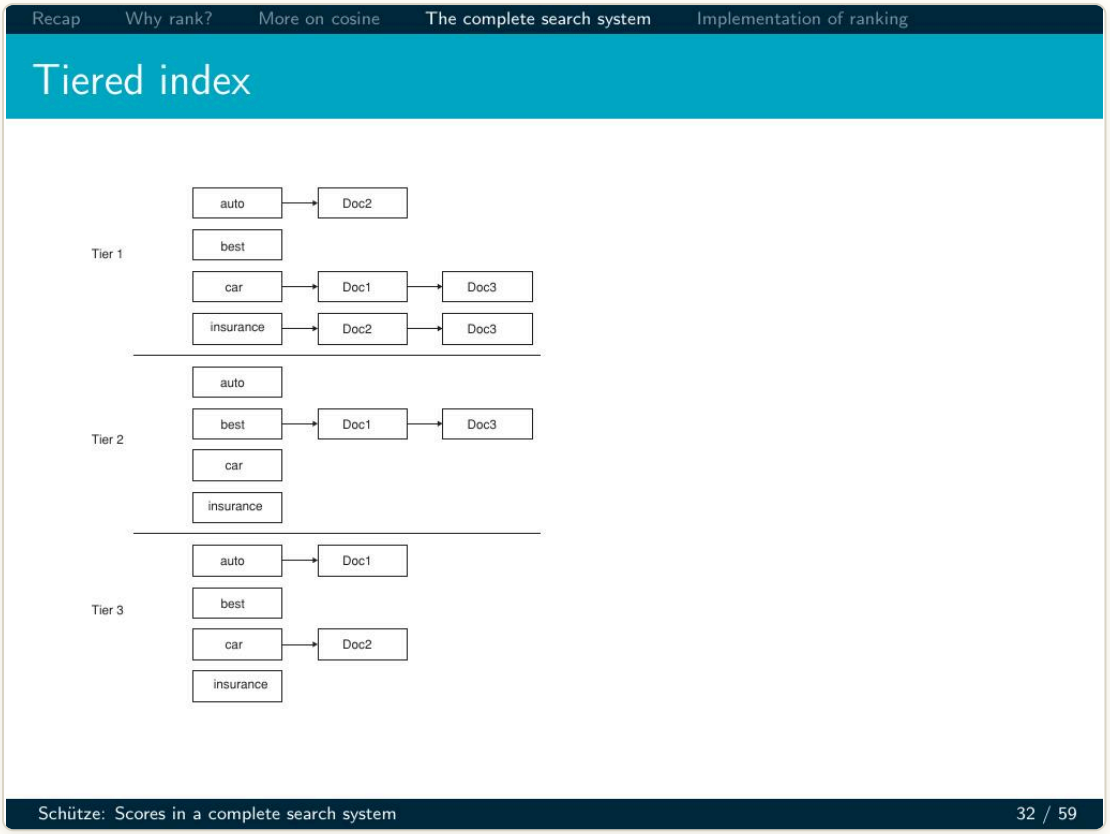


계층 색인 (Tiered indexes)

- 기본 아이디어
  - 색인 단어의 **중요도**에 따라 색인을 여러 **계층(tier)**으로
  - 질의 처리 때 **가장 높은 계층**부터
  - 최상위 계층이 **k개 이상**(예: k = 100) 결과를 내면: 멈추고 반환
  - k개 미만이면: 다음 계층에서 반복
- 예: 2계층 시스템
  - 계층 1: 모든 **제목**의 색인
  - 계층 2: 나머지 문서의 색인
- 검색어가 **제목**에 있는 페이지가 본문에 있는 페이지보다 더 좋은 결과

✍ 메모

★ 시험 계층 색인 = 중요한 것부터 찾고, k개 채우면 멈춤

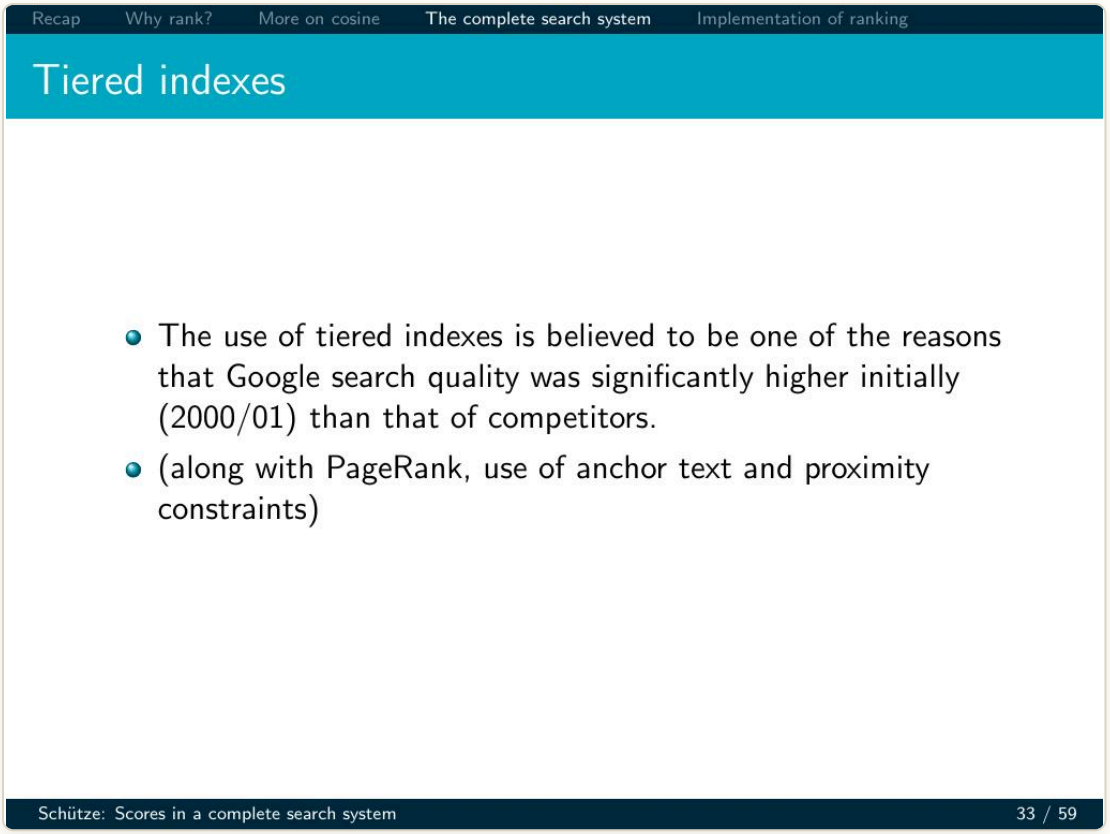


계층 색인 (그림)

- Tier 1: auto → Doc2 / best → (없음) / car → Doc1, Doc3 / insurance → Doc2, Doc3
- Tier 2: best → Doc1, Doc3 (나머지 비어 있음)
- Tier 3: auto → Doc1 / car → Doc2

✍️ 메모

— 넘어가도 되는 쪽

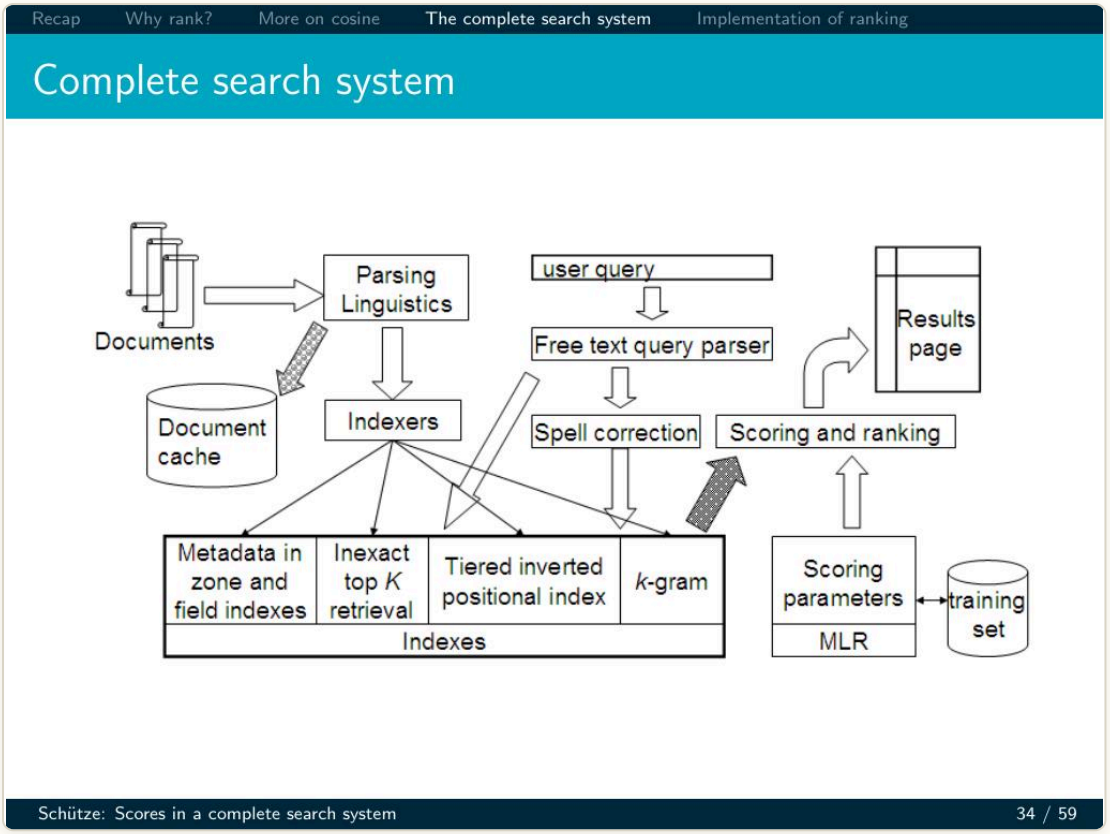


계층 색인과 구글

- 계층 색인은 초기(2000/01) 구글 검색 품질이 경쟁사보다 **훨씬 높았던** 이유 중 하나로 여겨진다
- (**PageRank**, **앵커 텍스트** 사용, **근접 제약**과 함께)

✍ 메모

— 넘어가도 되는 쪽

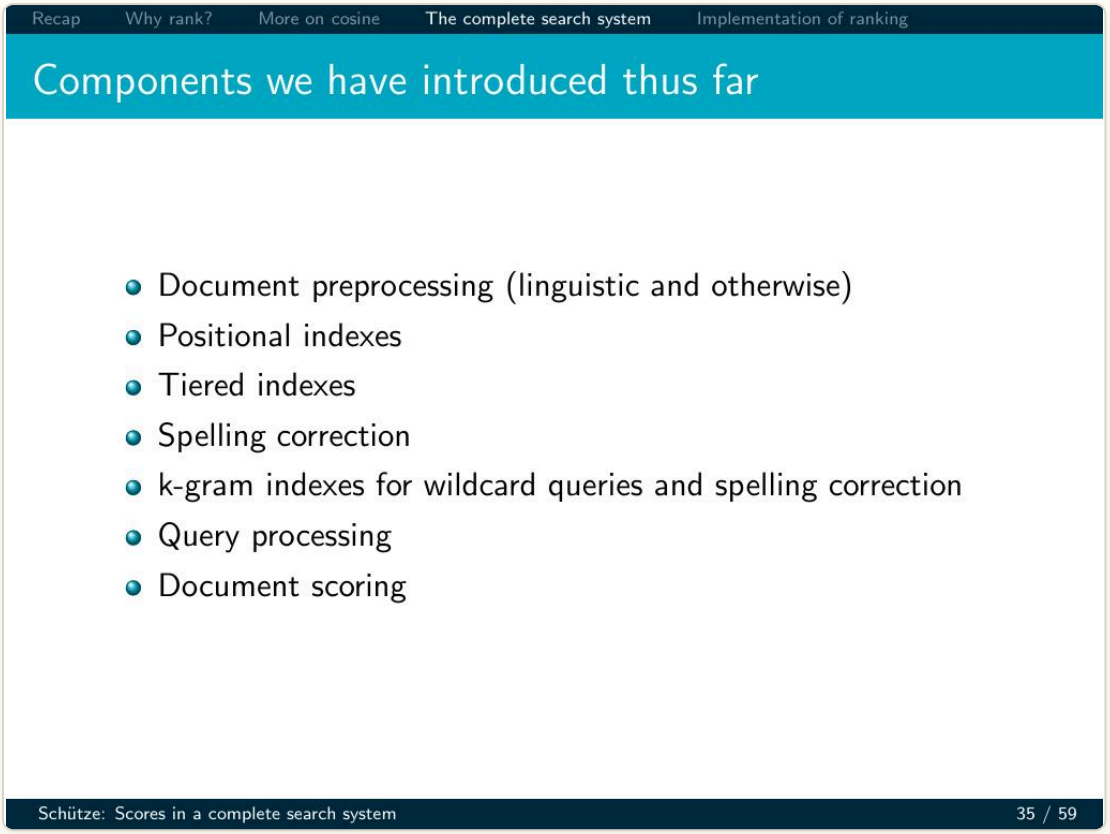


완전한 검색 시스템 (그림 다시)

- p.30과 같은 그림 — 지금까지 배운 것과 아직 안 배운 것을 나눠 보자

✍ 메모

— 넘어가도 되는 쪽

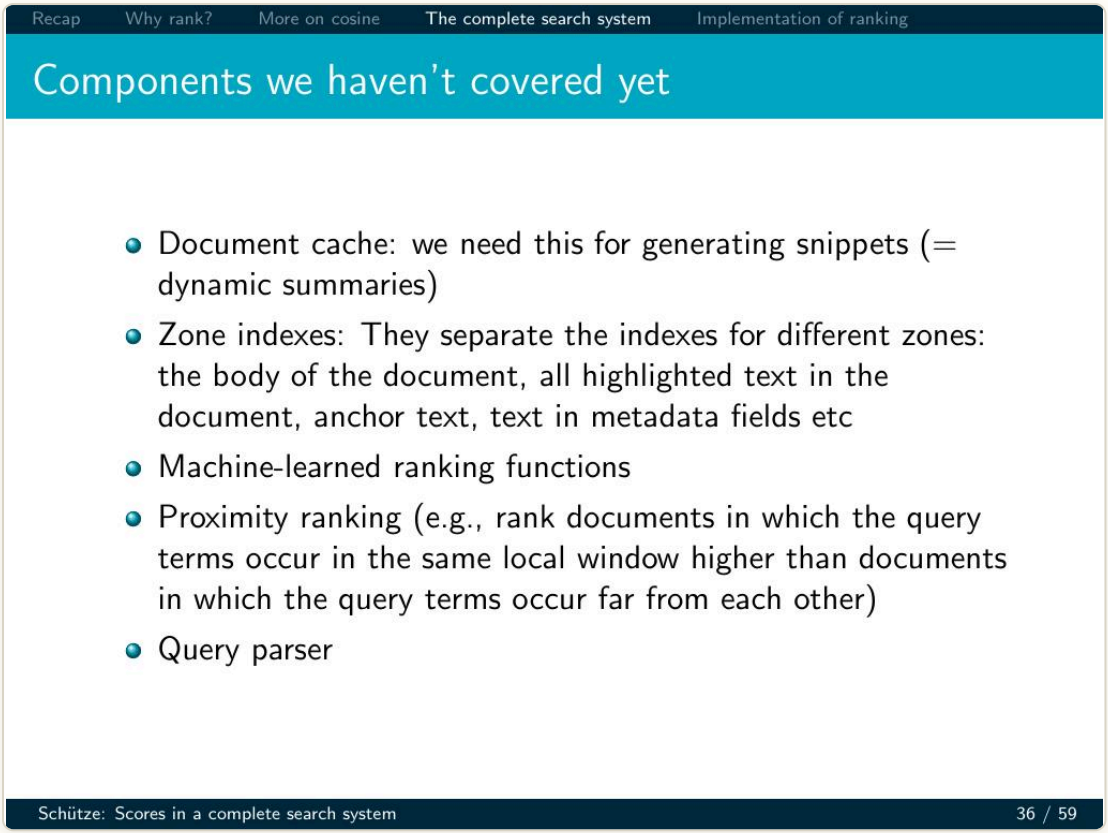


지금까지 소개한 구성 요소

- 문서 전처리 (언어적 처리 등)
- 위치 색인
- 계층 색인
- 철자 교정
- 와일드카드·철자 교정용 k-gram 색인
- 질의 처리
- 문서 점수화

✍ 메모

— 넘어가도 되는 쪽

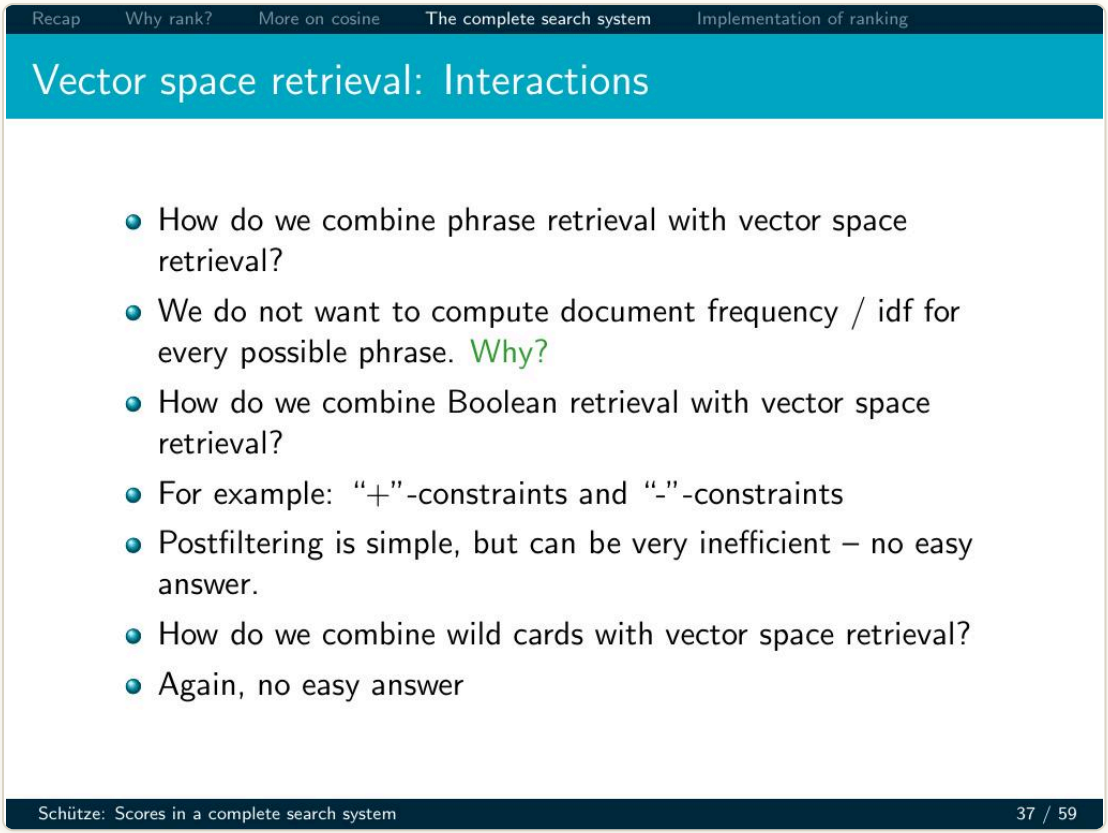


아직 다루지 않은 구성 요소

- **문서 캐시**: 스니펫(= 동적 요약)을 만드는 데 필요
- **영역 색인(Zone indexes)**: 문서의 영역별로 색인을 나눈다 — 본문, 강조 텍스트, 앵커 텍스트, 메타데이터 필드 텍스트 등
- **기계학습 순위 함수**
- **근접 순위**: 질의어가 같은 좁은 창 안에 나오는 문서를 멀리 떨어져 나오는 문서보다 위로
- **질의 파서**

✍ 메모

— 넘어가도 되는 쪽

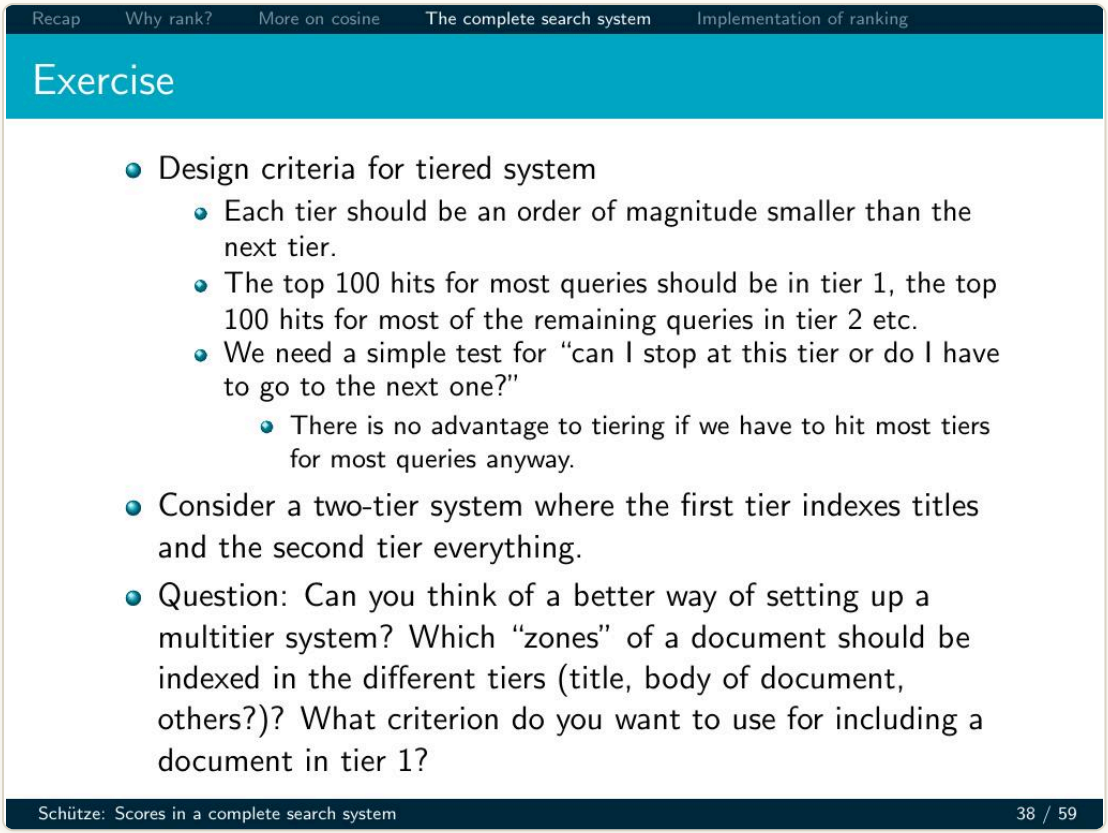


벡터 공간 검색: 상호작용

- **구 검색**을 벡터 공간 검색과 어떻게 합치나?
  - 가능한 모든 구에 대해 df/idf를 계산하고 싶진 않다. 왜?
- **불리언 검색**을 벡터 공간 검색과 어떻게 합치나?
  - 예: '+' 제약과 '−' 제약
  - 후처리(postfiltering)는 간단하지만 매우 비효율적일 수 있다 — 쉬운 답은 없다
- **와일드카드**를 벡터 공간 검색과 어떻게 합치나?
  - 역시 쉬운 답은 없다

✍️ 메모

💡 이해 모든 구의 df를 구하면 구의 종류가 너무 많다(어휘 폭증)

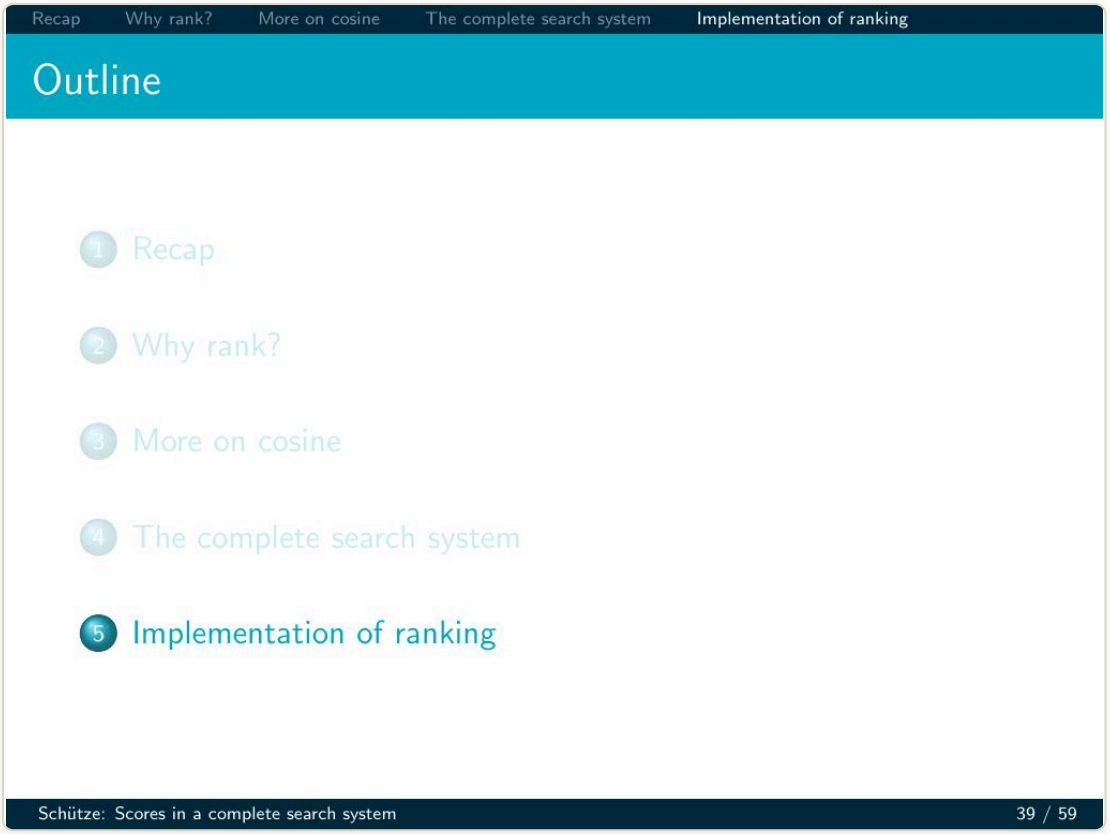


연습: 계층 시스템 설계 기준

- 각 계층은 다음 계층보다 **한 자릿수(10배) 작아야**
- 대부분 질의의 상위 100개가 계층 1에, 나머지 질의 대부분의 상위 100개가 계층 2에 ...
- '이 계층에서 멈춰도 되나, 다음으로 가야 하나?'를 판단할 **간단한 검사**가 필요
- 대부분 질의가 어차피 대부분 계층을 거쳐야 한다면 계층화는 **이점이 없다**
- 1계층은 제목, 2계층은 전부인 2계층 시스템을 생각하자
- 질문: 더 나은 다계층 구성은? 각 계층에 문서의 어떤 '영역'(제목, 본문, 기타)을 색인할까? 문서를 계층 1에 넣는 기준은?

✍️ 메모

— 넘어가도 되는 쪽

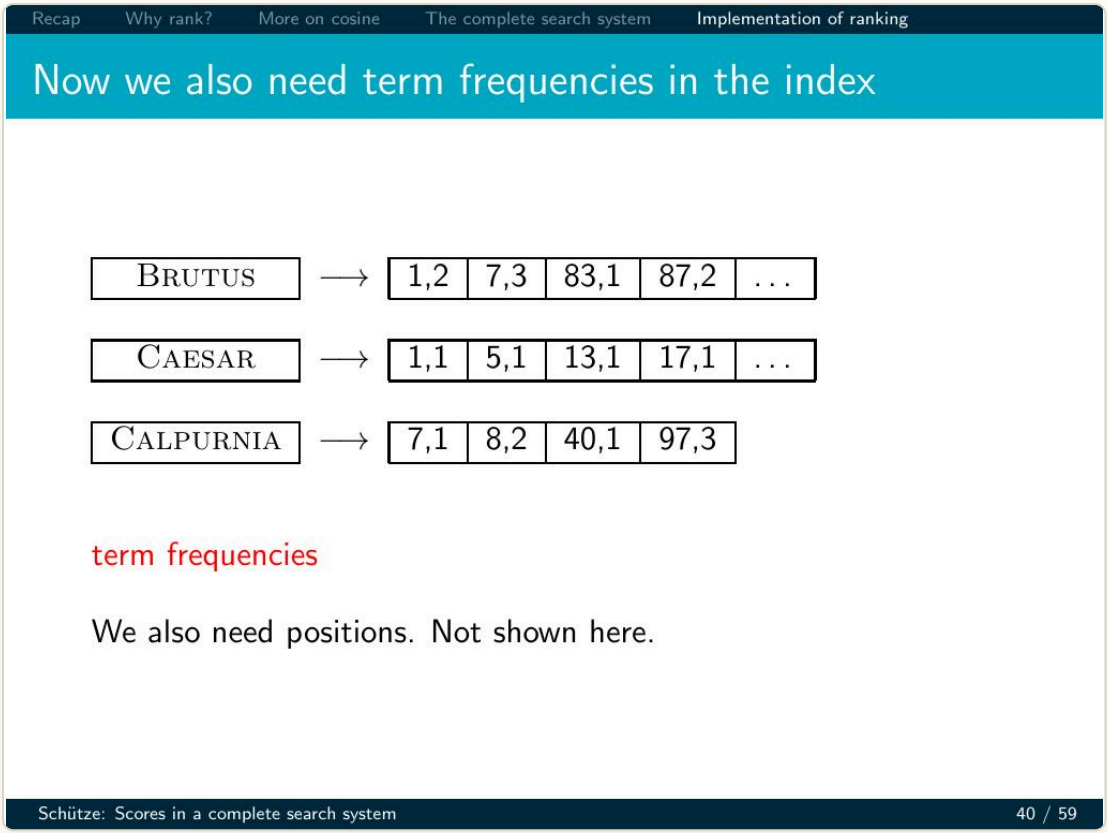


목차

- 복습 → 왜 순위가 중요한가 → 코사인 더 보기 → 완전한 검색 시스템 → 순위 구현
- 지금: **순위 구현**

✍ 메모

— 넘어가도 되는 쪽

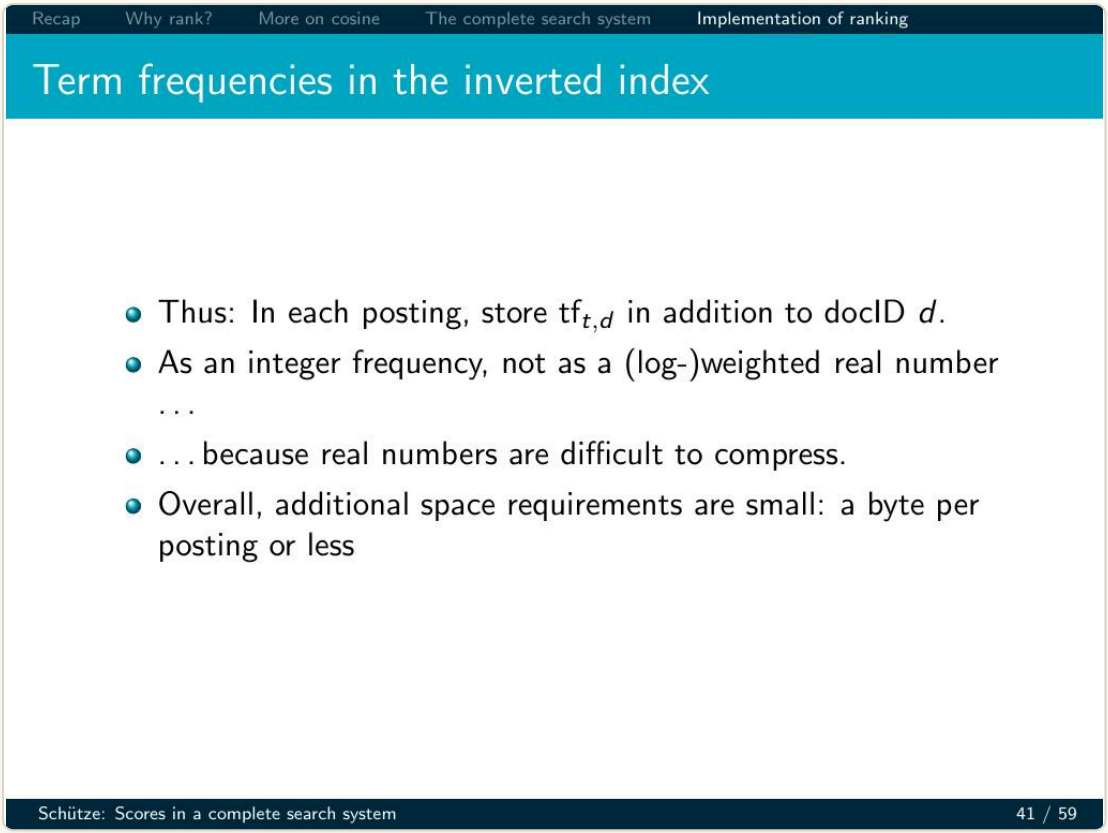


이제 색인에 단어 빈도도 필요하다

- Brutus → (1,2) (7,3) (83,1) (87,2) ...
- Caesar → (1,1) (5,1) (13,1) (17,1) ...
- Calpurnia → (7,1) (8,2) (40,1) (97,3)
- 포스팅 = (docID, **단어 빈도**)
- 위치도 필요하지만 여기선 생략

✍ 메모

— 넘어가도 되는 쪽

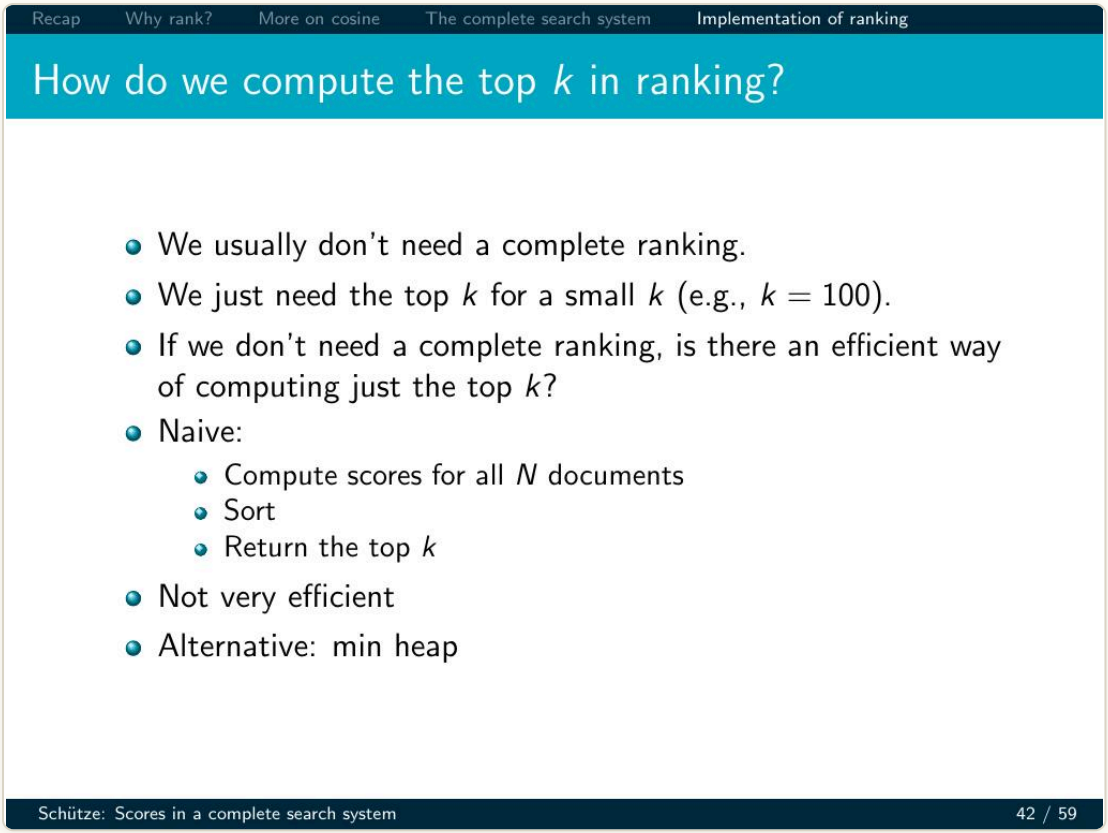


역색인 안의 단어 빈도

- 그래서 각 포스팅에 docID d와 함께 **tf(t,d)**를 저장
- (로그) 가중치 실수가 아니라 **정수 빈도**로 ...
- ... 실수는 **압축하기 어려우니까**
- 전체 추가 공간은 작다: 포스팅당 **1바이트 이하**

✍ 메모

★ 시험 tf는 정수로 저장 (실수는 압축이 어려움)

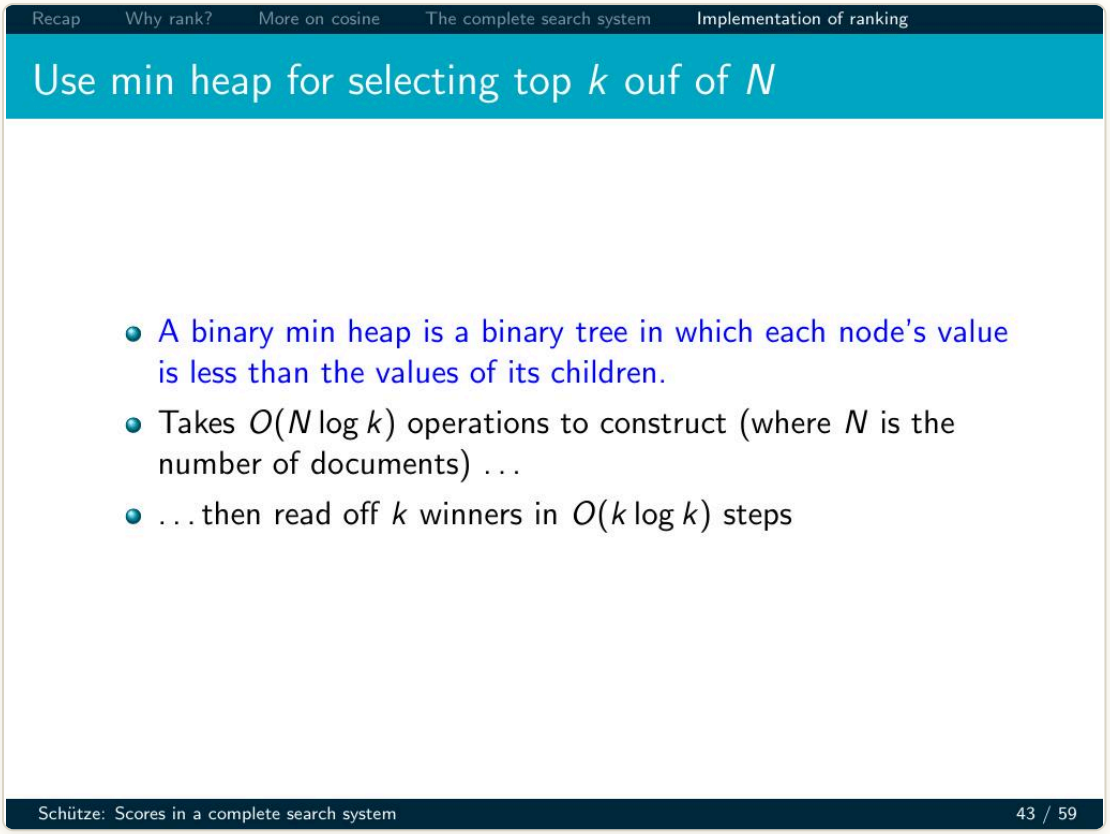


순위에서 top k를 어떻게 구하나?

- 보통 **전체 순위**는 필요 없다
- 작은 k(예: k = 100)의 **top k**만 필요
- 전체 순위가 필요 없다면 top k만 효율적으로 구하는 방법이 있을까?
- 단순한 방법: N개 문서 전부 점수 계산 → 정렬 → 상위 k 반환
  - 그리 효율적이지 않다
- 대안: **최소 힙(min heap)**

✍️ 메모

— 넘어가도 되는 쪽

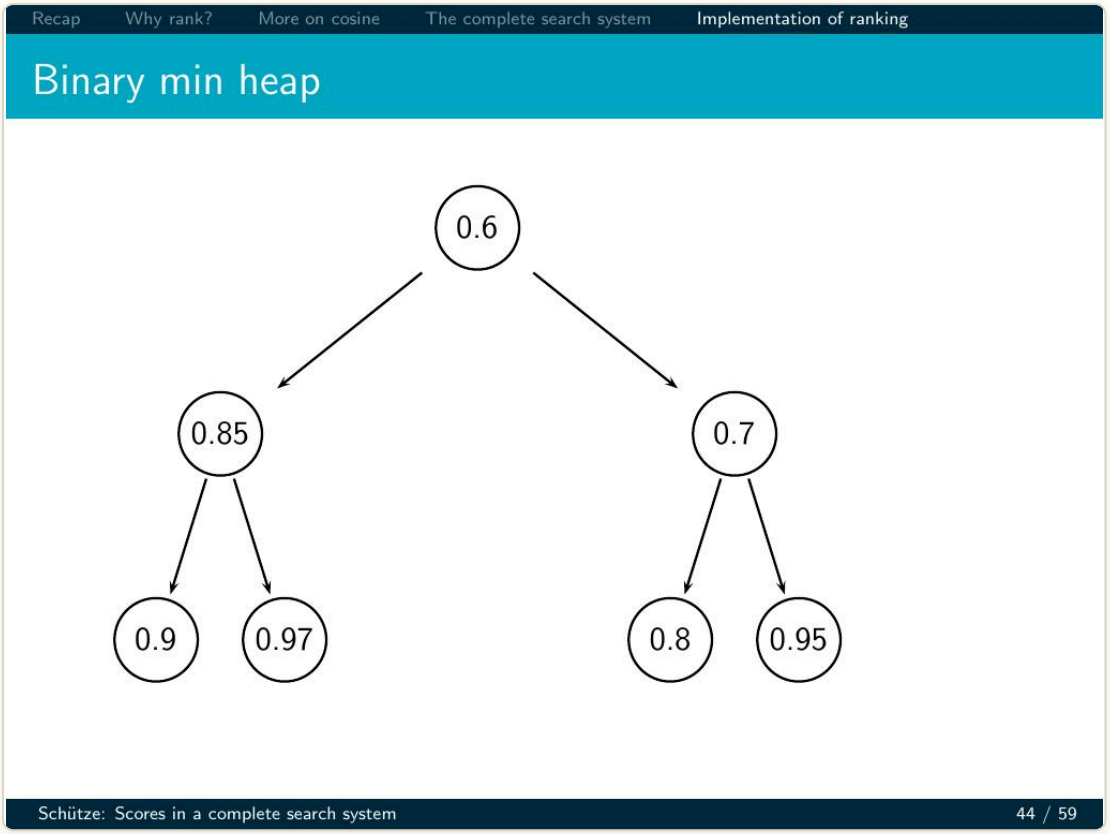


최소 힙으로 N개 중 top k 고르기

- **이진 최소 힙** = 각 노드의 값이 **자식보다 작은** 이진 트리
- 구성에 **O(N log k)** 연산 (N = 문서 수) ...
- ... 그다음 승자 k개를 **O(k log k)** 단계로 읽어 낸다

✍ 메모

— 넘어가도 되는 쪽

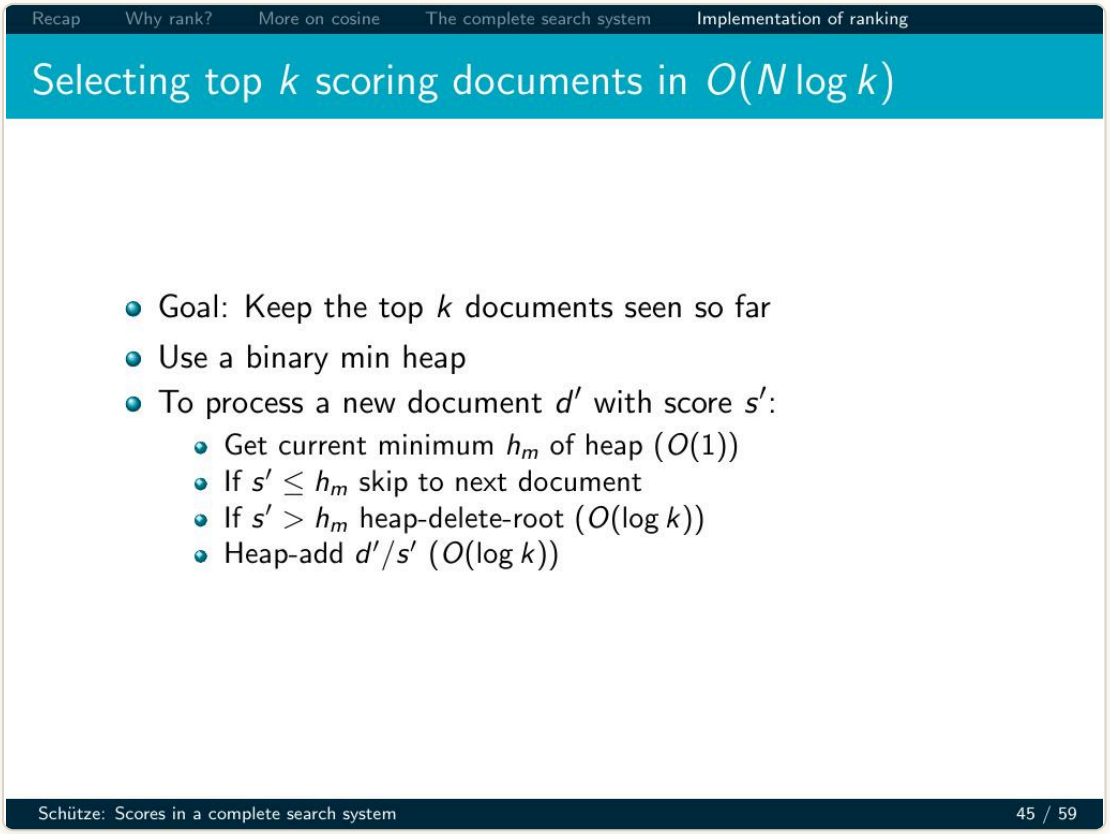


이진 최소 힙 (그림)

- 루트 0.6 → 자식 0.85, 0.7 → 손자 0.9, 0.97 / 0.8, 0.95
- 루트가 항상 **최솟값** = 지금 top k 중 가장 낮은 점수

✍ 메모

— 넘어가도 되는 쪽

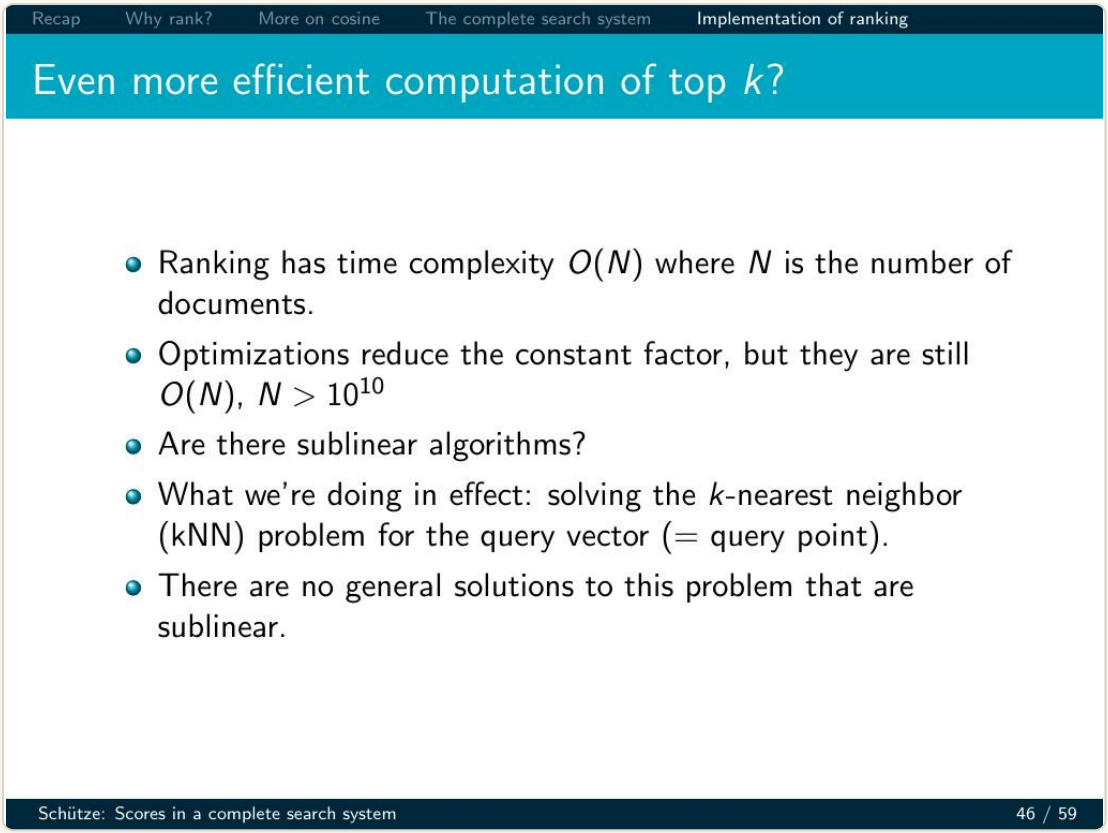


O(N log k)로 top k 문서 고르기

- 목표: 지금까지 본 문서 중 **top k**를 유지
- 이진 최소 힙 사용
- 점수 s′인 새 문서 d′ 처리
  - 힙의 현재 최솟값 hm 확인 — **O(1)**
  - s′ ≤ hm이면 다음 문서로 건너뜀
  - s′ > hm이면 루트 삭제 — **O(log k)**
  - d′/s′ 추가 — **O(log k)**

✍ 메모

★ 시험 힙 크기를 k로 유지 → 문서마다 O(log k) → 전체 O(N log k)

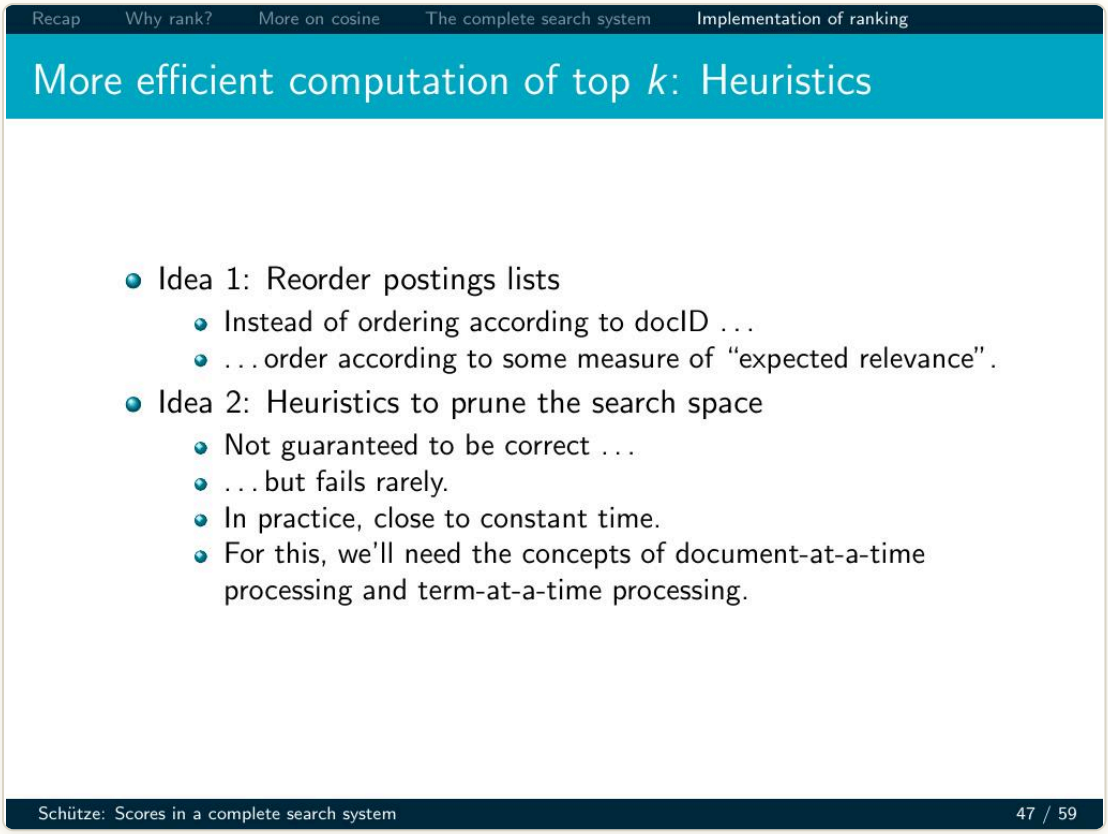


top k를 더 효율적으로?

- 순위의 시간 복잡도는 **O(N)** (N = 문서 수)
- 최적화는 상수 배를 줄일 뿐, 여전히 O(N), N > 10^{10}
- **준선형(sublinear)** 알고리즘이 있을까?
- 사실상 하는 일: 질의 벡터(= 질의 점)에 대한 **k-최근접 이웃(kNN)** 문제 풀기
- 이 문제엔 **일반적인 준선형 해법이 없다**

✍ 메모

— 넘어가도 되는 쪽

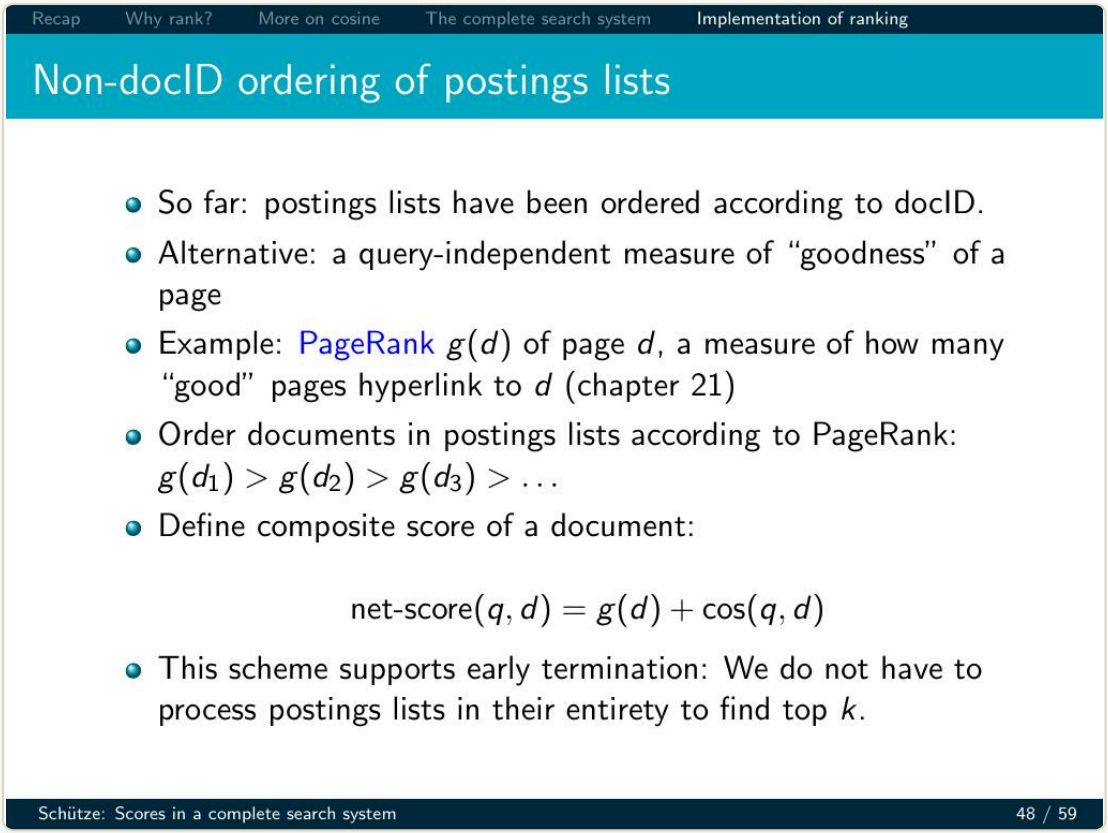


top k를 더 효율적으로: 휴리스틱

- 아이디어 1: 포스팅 리스트 **재정렬**
  - docID순 대신 ...
  - ... '예상 관련성'의 어떤 척도순으로
- 아이디어 2: 탐색 공간을 **가지치기**하는 휴리스틱
  - 정확성이 보장되진 않지만 ...
  - ... 거의 실패하지 않는다
  - 실제로는 **거의 상수 시간**
- 이를 위해 **문서 단위(document-at-a-time)** 처리와 **단어 단위(term-at-a-time)** 처리 개념이 필요

✍ 메모

— 넘어가도 되는 쪽

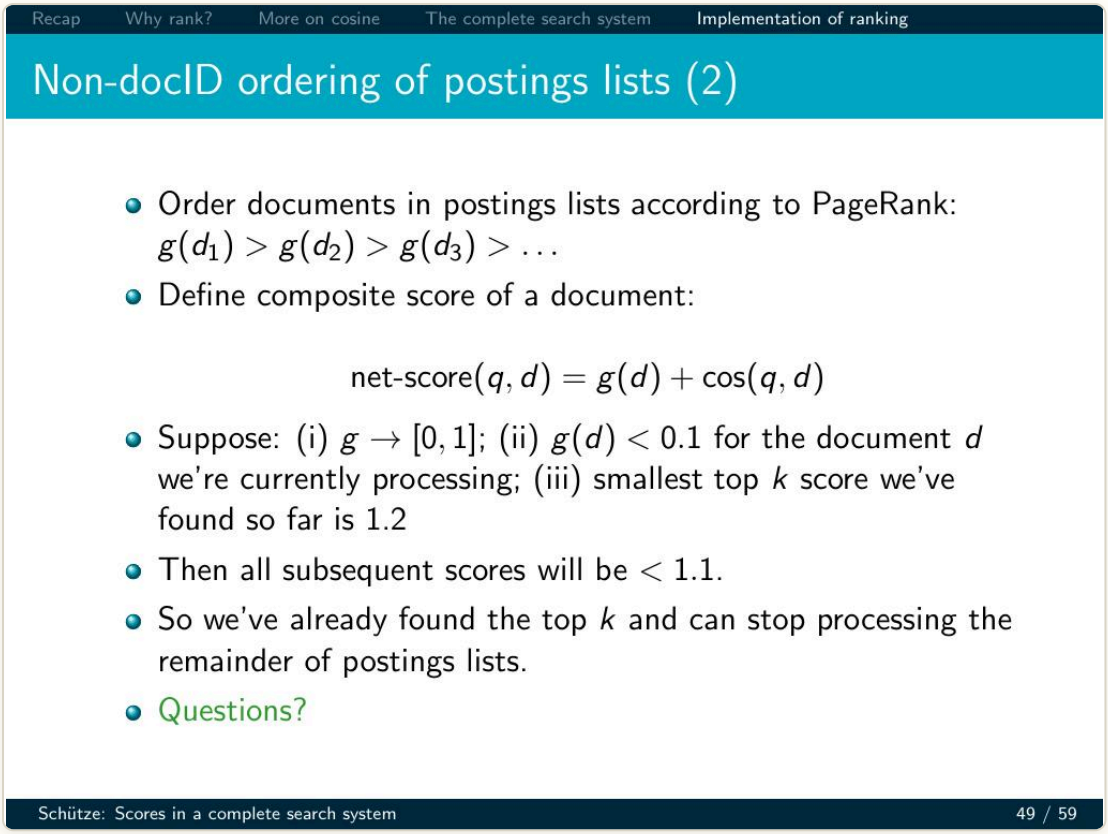


docID가 아닌 순서의 포스팅 리스트

- 지금까지 포스팅 리스트는 docID순
- 대안: 페이지의 **질의와 무관한 '좋음'** 척도
  - 예: 페이지 d의 **PageRank g(d)** — 얼마나 많은 '좋은' 페이지가 d로 링크하는가 (21장)
- 포스팅 리스트의 문서를 PageRank순으로: $g(d_1) > g(d_2) > g(d_3) > \ldots$
- 문서의 복합 점수: **net-score(q, d) = g(d) + cos(q, d)**
- 이 방식은 **조기 종료**를 지원: top k를 찾으려고 포스팅 리스트를 끝까지 볼 필요가 없다

✍ 메모

— 넘어가도 되는 쪽

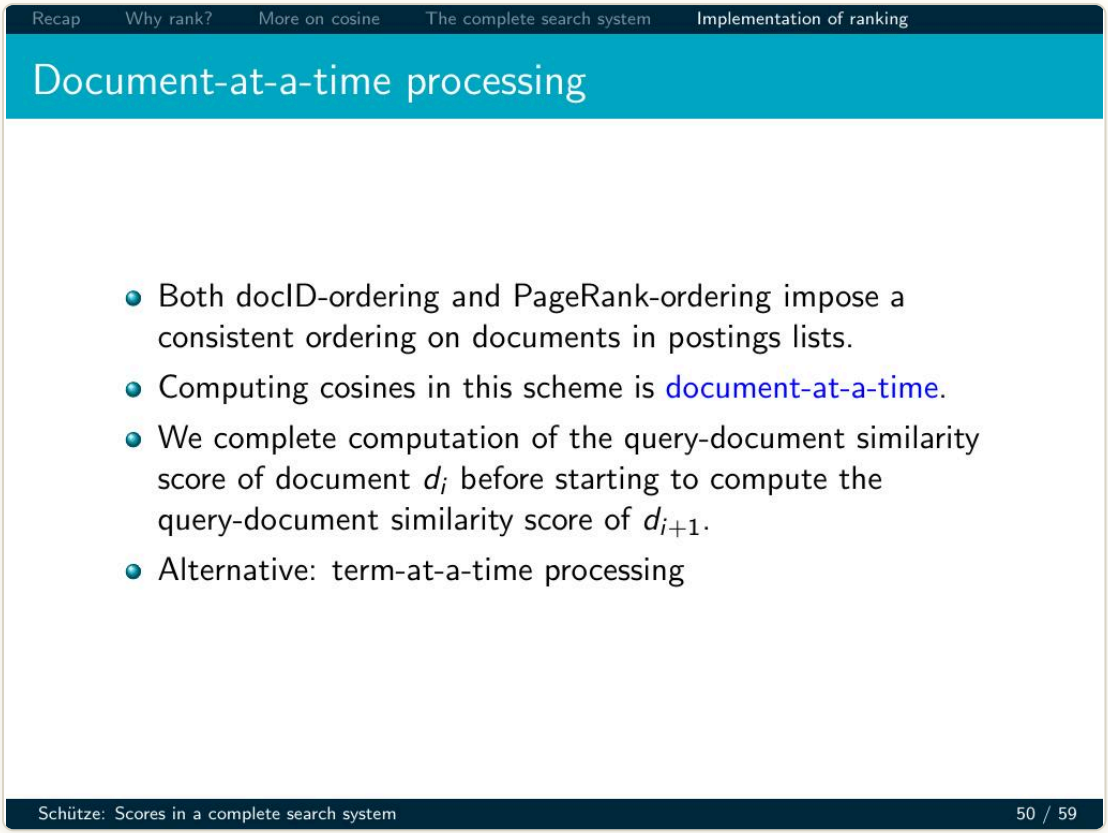


docID가 아닌 순서의 포스팅 리스트 (2)

- PageRank순 정렬, net-score(q, d) = g(d) + cos(q, d)
- 가정: (i) g → [0, 1] (ii) 지금 처리 중인 문서 d의 g(d) < 0.1 (iii) 지금까지 찾은 top k 중 가장 낮은 점수가 1.2
- 그러면 이후 모든 점수는 **< 1.1** (g < 0.1, cos ≤ 1)
- 그러니 이미 top k를 찾았고, 나머지 포스팅 처리는 **멈춰도 된다**

✍ 메모

✏ 예·풀이 뒤에 올 최대 점수 = 0.1 + 1 = 1.1 < 1.2 → 조기 종료

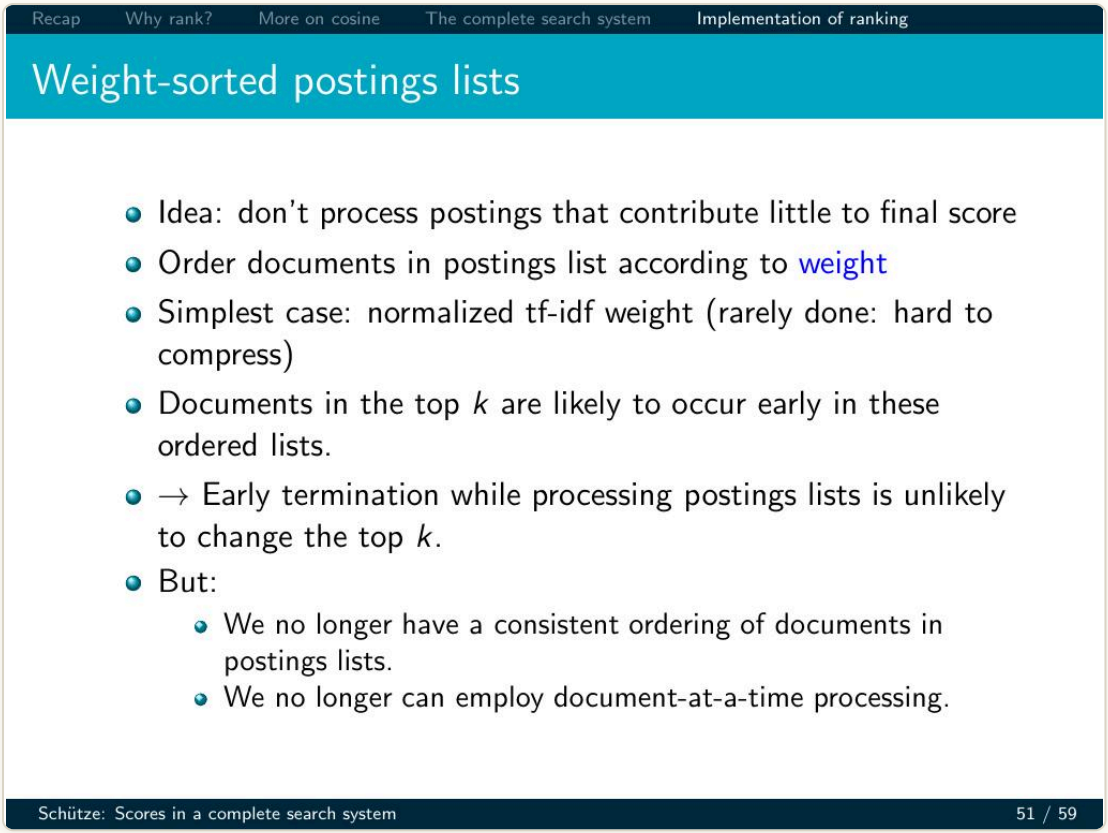


문서 단위 처리 (Document-at-a-time)

- docID순과 PageRank순은 모두 포스팅 리스트의 문서에 **일관된 순서**를 준다
- 이 방식에서 코사인 계산은 **문서 단위**
- 문서 d_i의 질의-문서 유사도 계산을 **끝낸 뒤** d_{i+1}을 시작
- 대안: **단어 단위** 처리

✍ 메모

— 넘어가도 되는 쪽

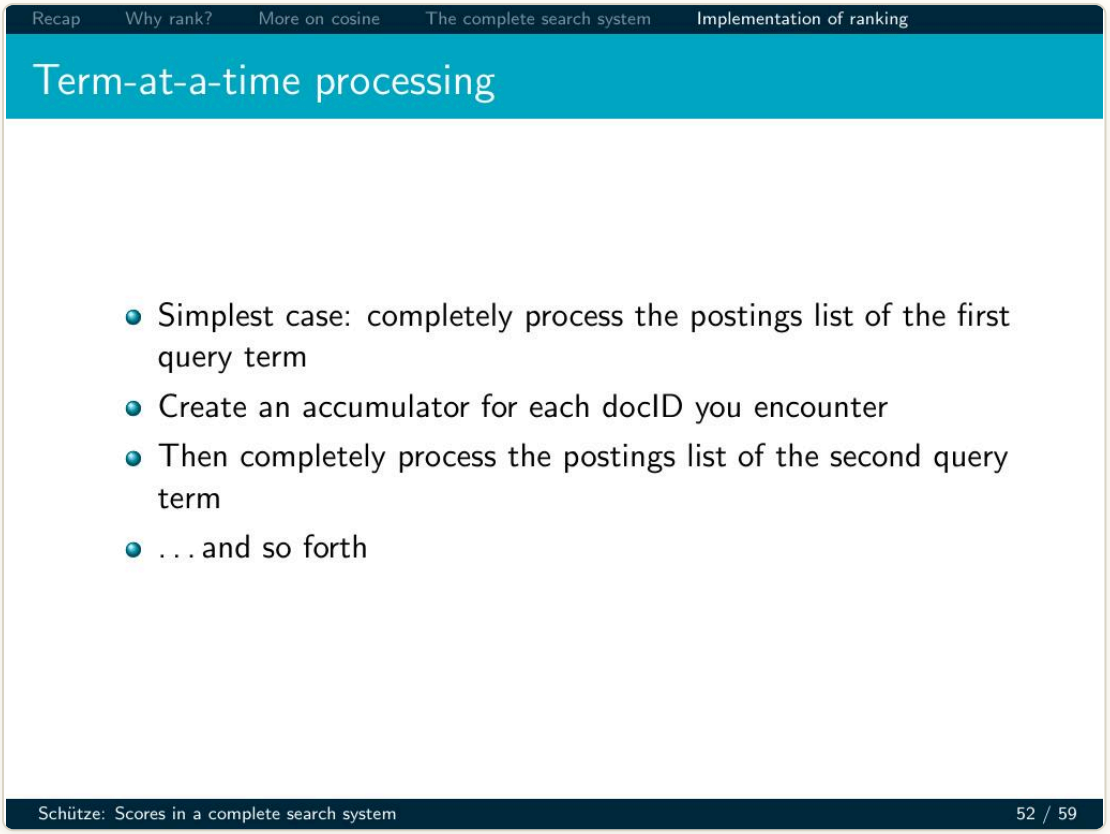


가중치순 포스팅 리스트

- 아이디어: 최종 점수에 **적게 기여하는** 포스팅은 처리하지 않는다
- 포스팅 리스트의 문서를 **가중치순**으로
- 가장 단순한 경우: 정규화된 tf-idf 가중치 (압축이 어려워 드물게 씀)
- top k 문서는 이렇게 정렬된 리스트의 **앞부분**에 나올 가능성이 크다
- → 포스팅 리스트 처리를 일찍 멈춰도 top k가 바뀔 가능성은 낮다
- 하지만
  - 포스팅 리스트의 문서 순서가 더 이상 **일관되지 않다**
  - **문서 단위 처리를 쓸 수 없다**

✍ 메모

★ 시험 가중치순 정렬 → 단어마다 순서가 달라짐 → term-at-a-time만 가능

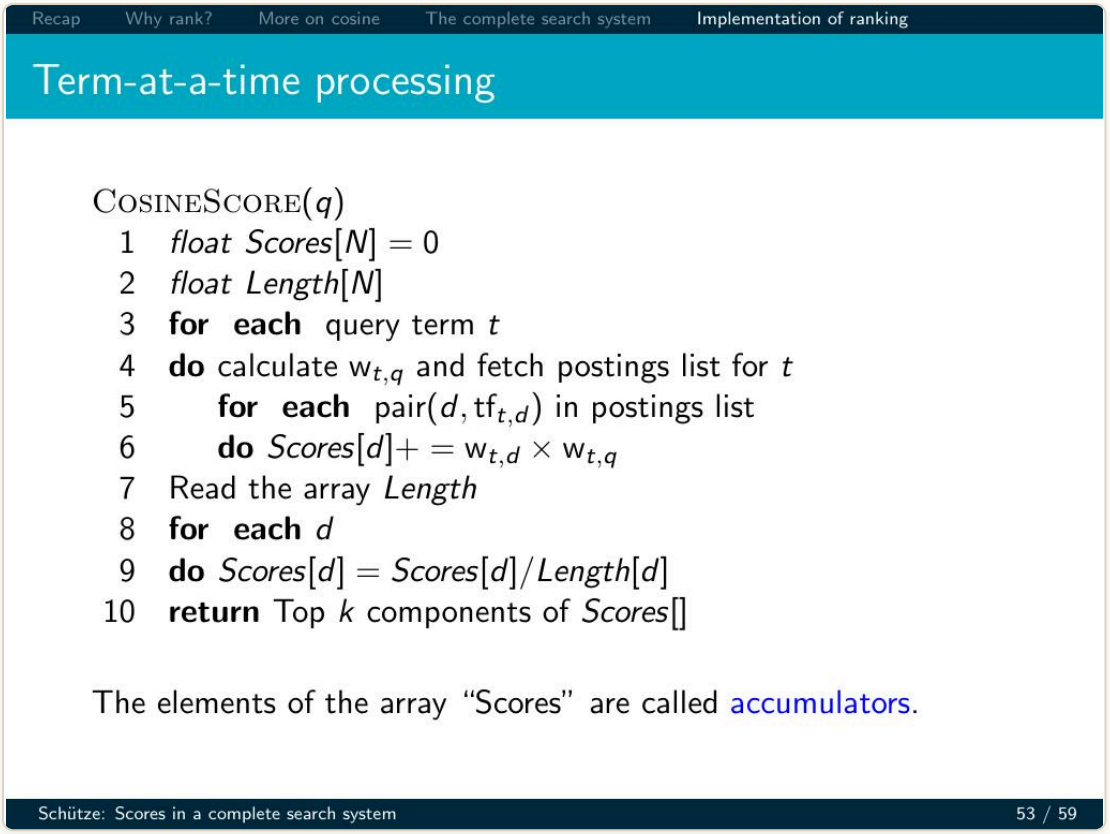


단어 단위 처리 (Term-at-a-time)

- 가장 단순한 경우: 첫 질의어의 포스팅 리스트를 **완전히** 처리
  - 만나는 docID마다 **누산기(accumulator)**를 만든다
- 그다음 둘째 질의어의 포스팅 리스트를 완전히 처리
- ... 이런 식으로

✍ 메모

— 넘어가도 되는 쪽



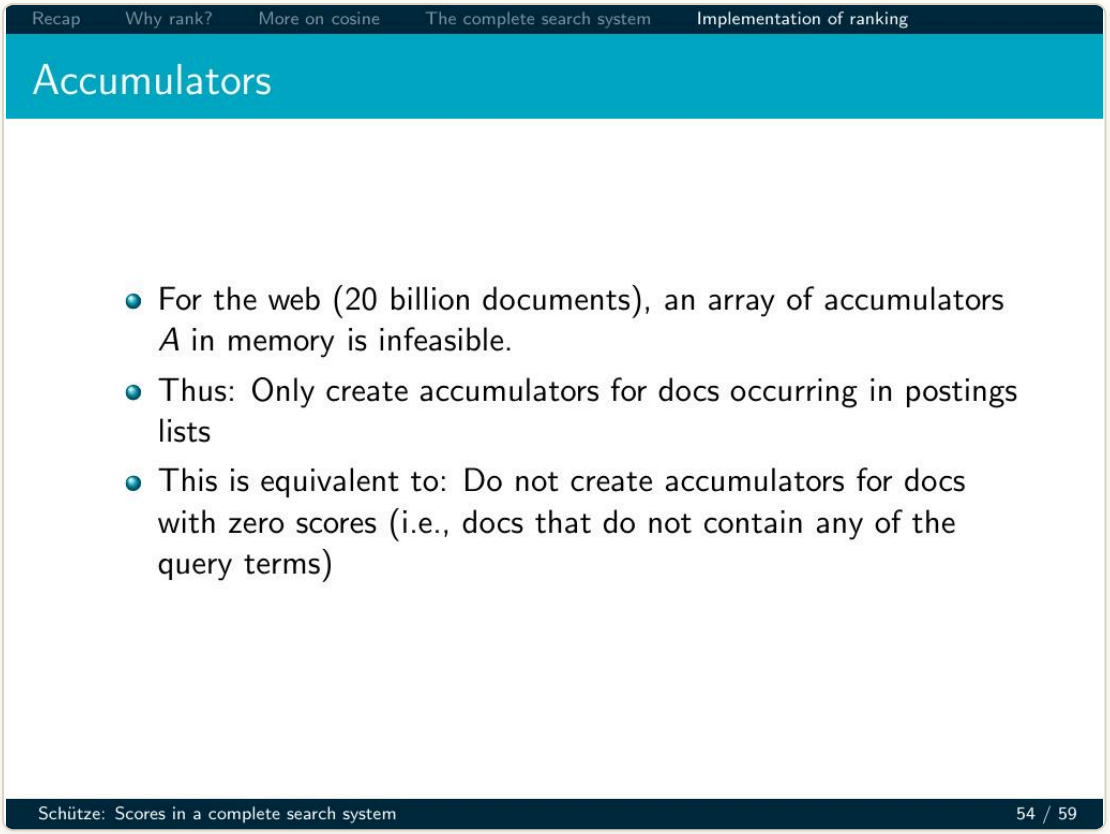
단어 단위 처리: COSINESCORE

```
float Scores[N] = 0; float Length[N]
for each 질의어 t: w(t,q) 계산, 포스팅 리스트 가져오기
  for each (d, tf(t,d)): Scores[d] += w(t,d) × w(t,q)
Length 읽기; for each d: Scores[d] /= Length[d]
return Scores[]의 top k
```

- 배열 'Scores'의 원소를 **누산기**라 한다

✍ 메모

— 넘어가도 되는 쪽

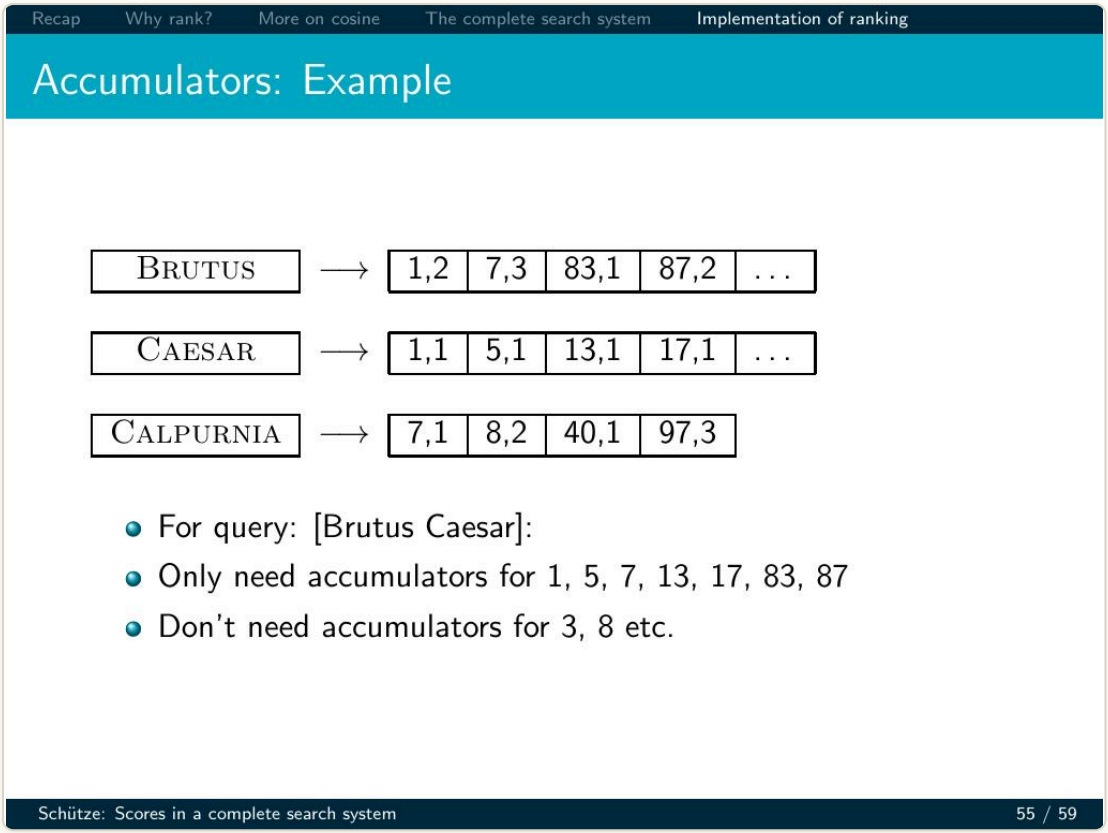


누산기 (Accumulators)

- 웹(문서 200억 개)에선 메모리에 누산기 배열 A를 두는 게 **불가능**
- 그래서 **포스팅 리스트에 나오는 문서**에만 누산기를 만든다
- 같은 말: **점수가 0인 문서**(질의어가 하나도 없는 문서)엔 누산기를 만들지 않는다

✍️ 메모

— 넘어가도 되는 쪽

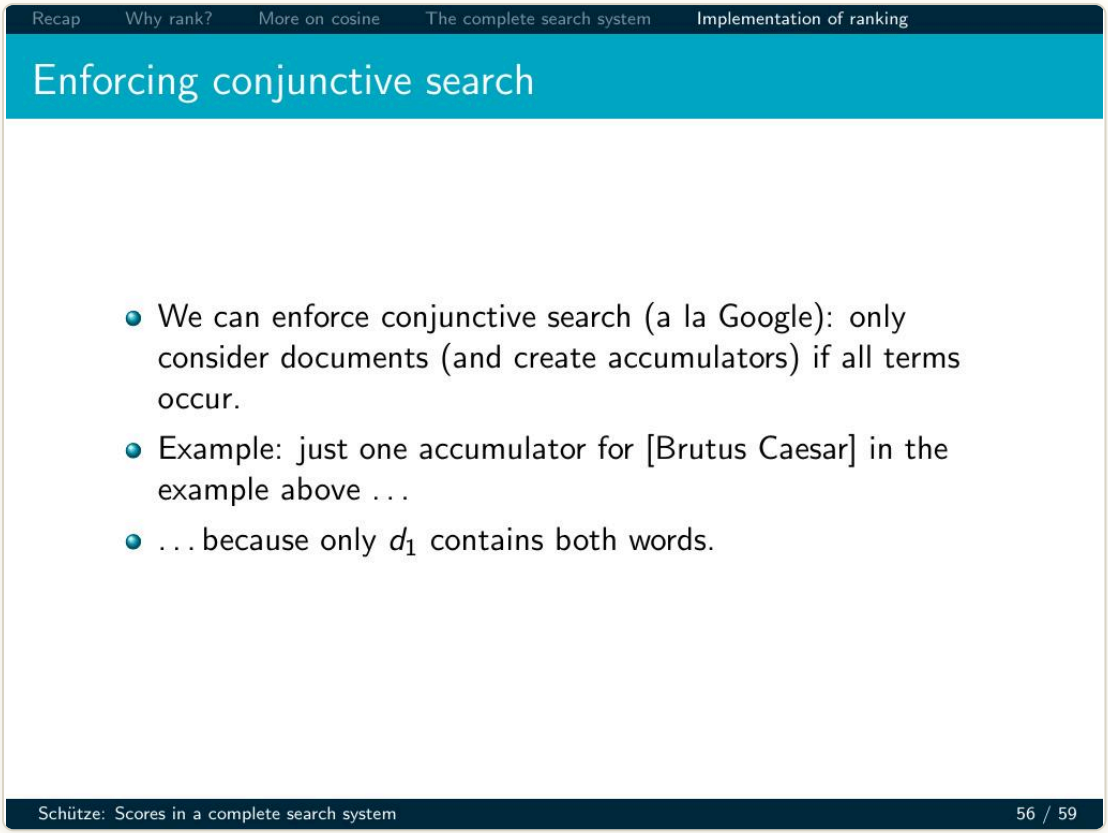


누산기: 예

- Brutus → (1,2) (7,3) (83,1) (87,2) ... / Caesar → (1,1) (5,1) (13,1) (17,1) ... / Calpurnia → (7,1) (8,2) (40,1) (97,3)
- 질의 [Brutus Caesar]
  - 누산기가 필요한 문서: **1, 5, 7, 13, 17, 83, 87**
  - 3, 8 등에는 필요 없다

✍ 메모

✏ 예·풀이 Brutus {1, 7, 83, 87} ∪ Caesar {1, 5, 13, 17} = 7개 (Calpurnia의 8, 40, 97은 질의에 없음)

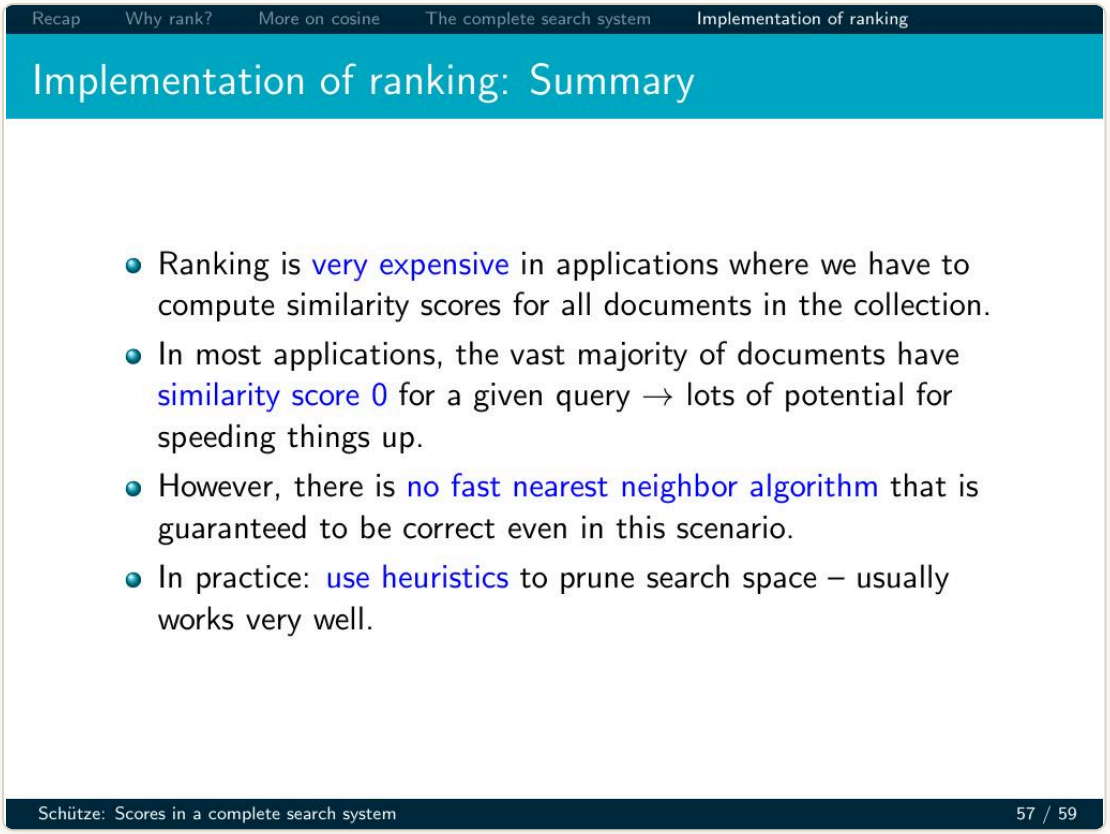


AND 검색 강제하기

- (구글처럼) **AND 검색**을 강제할 수 있다: **모든 단어가 나오는** 문서만 고려하고 누산기를 만든다
- 예: 위 예에서 [Brutus Caesar]엔 누산기가 **하나**뿐 ...
- ... 두 단어를 다 가진 문서는 **d1**뿐이니까

✍ 메모

— 넘어가도 되는 쪽

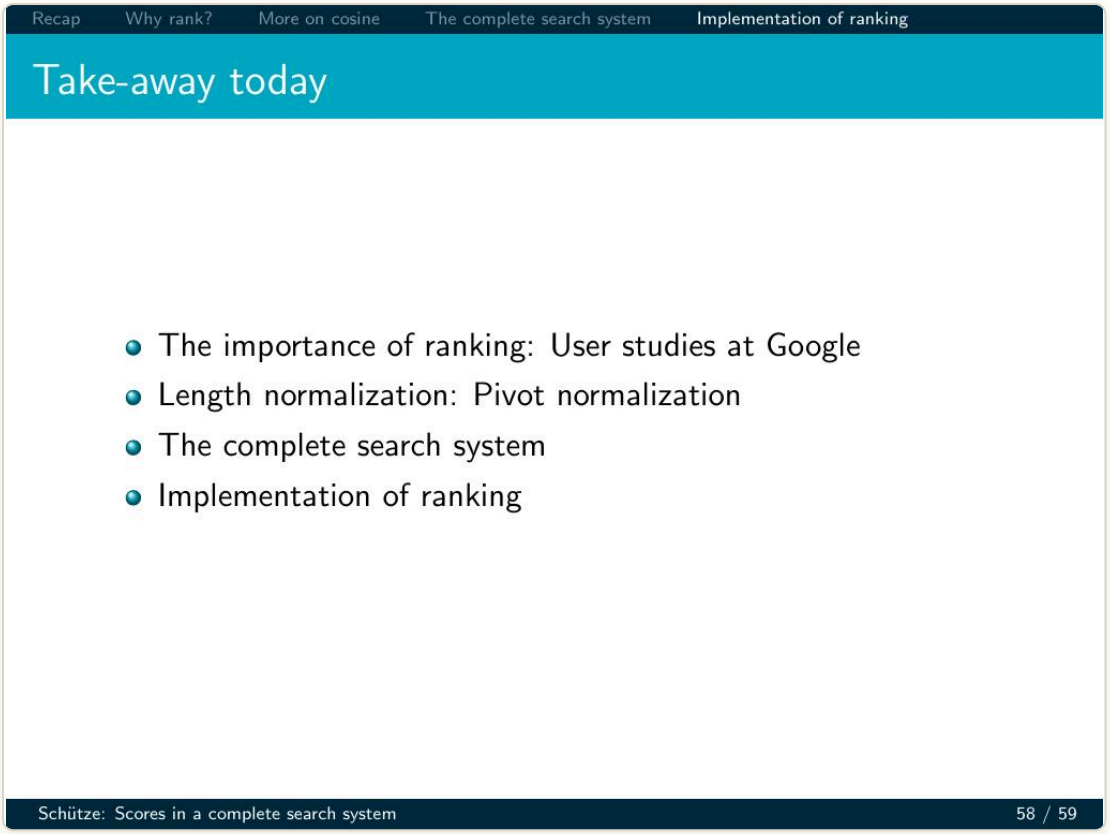


순위 구현: 요약

- 컬렉션의 **모든 문서**에 대해 유사도를 계산해야 하는 응용에선 순위가 **매우 비싸다**
- 대부분의 응용에서 주어진 질의에 대해 **대다수 문서의 유사도는 0** → 속도를 높일 여지가 많다
- 하지만 이 경우에도 **정확성이 보장되는 빠른 최근접 이웃 알고리즘은 없다**
- 실제로는: 탐색 공간을 가지치기하는 **휴리스틱** — 보통 아주 잘 된다

✍ 메모

— 넘어가도 되는 쪽

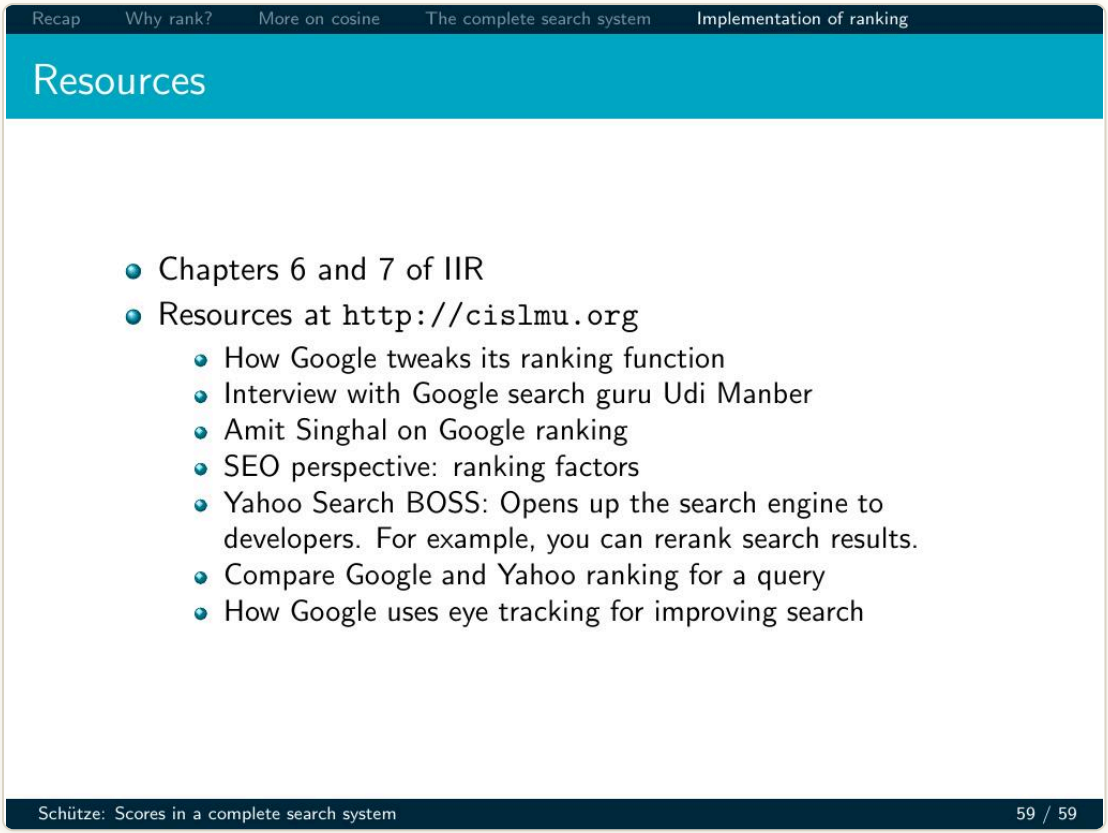


오늘 가져갈 것

- 순위의 중요성: 구글 사용자 연구
- 길이 정규화: 피벗 정규화
- 완전한 검색 시스템
- 순위 구현 (최소 힙, 조기 종료, 누산기)

✍ 메모

— 넘어가도 되는 쪽



참고 자료

- IIR 6·7장, http://cislmu.org
- 구글의 순위 함수 조정, 구글 검색 전문가 Udi Manber 인터뷰, Amit Singhal의 구글 순위
- SEO 관점의 순위 요인, Yahoo Search BOSS, 구글과 야후 순위 비교, 구글의 시선 추적 활용

✍️ 메모

— 넘어가도 되는 쪽